

國立臺灣大學工學院工業工程學研究所

碩士論文

Department of Industrial Engineering

College of Engineering

National Taiwan University

Master's Thesis

以條件流匹配加速區域尺度 StormCast：應用於臺灣對
流尺度天氣預報的單階段流匹配殘差生成模型

Accelerating Regional StormCast with Conditional Flow
Matching:

A Single-Run Flow-Matching Residual Forecaster for
Taiwan Convective-Scale Weather Prediction

Desmond Cheong

Desmond Cheong

指導教授: Advisor Name 博士

Advisor: Advisor Name, Ph.D.

中華民國 115 年 6 月

June 2026



國立臺灣大學碩士學位論文

口試委員會審定書



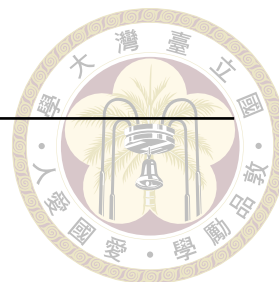
以條件流匹配加速區域尺度 StormCast：應用於
臺灣對流尺度天氣預報的單階段流匹配殘差生成
模型

Accelerating Regional StormCast with Conditional
Flow Matching:
A Single-Run Flow-Matching Residual Forecaster
for
Taiwan Convective-Scale Weather Prediction

本論文係 Desmond Cheong 君 (R00000000) 在國立臺灣大學工業工程學研究所完成之碩士學位論文，於民國 115 年 6 月 4 日承下列考試委員審查通過及口試及格，特此證明

口試委員：_____

(指導教授)



_____	_____
_____	_____
_____	_____
_____	_____

所 長：_____



Acknowledgements

I would like to thank my advisor for guidance throughout this thesis, and the members of the lab for the many discussions that shaped this work. I am grateful to the developers of StormCast and PhysicsNeMo for releasing the pretrained checkpoints and training infrastructure that made this study possible, and to the Taiwan Central Weather Administration / NCDR for the RWRF reanalysis data used as training targets. Finally, I thank my family for their support throughout the program.





摘要

近年來，基於擴散模型的生成式天氣預報（例如 GenCast 與 StormCast）已展現出在機率性、銳利化、且可集成的短時預報任務上超越傳統確定性深度學習模型的能力。然而，擴散模型在推論階段需要沿著機率流常微分方程（PF-ODE）執行數十次神經網路前向傳遞（Number of Function Evaluations, NFE），使其在進行長時間自迴歸滾動預測或大規模集成預報時的計算成本過高；同時，擴散殘差在稀疏、重尾的降水通道上仍偏向平滑，難以重現對流尺度的極端降水結構。

本論文以一個以臺灣區域（RWRF）資料集訓練的兩階段 StormCast 模型（凍結的確定性回歸均值加上生成式殘差頭）為基礎，提出並評估 **FlowCast**：一個以條件流匹配（Conditional Flow Matching, CFM）為核心、無需教師、僅需單一訓練階段的殘差生成模型。FlowCast 透過迴歸一條由雜訊到資料的直線速度場 v_θ ，在推論時僅以少數幾步顯式 Euler 積分即可取樣，與需要約 25 NFE 的 EDM 擴散模型（保留為對照基準）形成對比。針對臺灣 RWRF 目標僅有 4 個高解析度輸出通道（t2m, u10, v10, qpepre）的特性，我們設計逐通道加權的 L2 損失，並在稀疏重尾的降水通道 qpepre 上加入徑向對數功率譜密度（log-PSD）正則化項；我們另外比較了 log1p 降水編碼與原始 mm/h 編碼，以及降水通道權重 w_{pr} 的取捨。

我們在 2022 年整年逐小時驗證資料上，以確定性誤差（RMSE/MAE）、機率



分數 (CRPS)、降水類別分數 (CSI、FSS、HSS、誤報率)、徑向功率譜，以及最長 12 小時的自迴歸滾動誤差等指標，將 FlowCast 與 EDM 擴散基準進行同訓練樣本預算下的正面比較。實驗結果顯示，在約三分之一的推論時間下，FlowCast 在 CRPS、風場 RMSE、降水 RMSE 與誤報率上即可達到或優於 EDM 擴散，並提供更佳的 NFE/時間對品質權衡。本研究為區域對流尺度生成式預報提供了一套可重現的單階段流匹配流程與降水導向的評估基準。

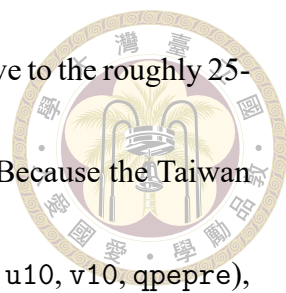
關鍵字：流匹配、條件流匹配、擴散模型、生成式天氣預報、對流尺度預報、降水臨近預報、StormCast



Abstract

Diffusion-based generative weather forecasters such as GenCast and StormCast have recently surpassed deterministic deep-learning baselines on probabilistic, sharp, and ensemble-capable short-range prediction tasks. However, sampling a diffusion model requires integrating its probability-flow ODE with tens of neural-network function evaluations (NFE) per forecast step, which makes autoregressive rollouts and large ensembles computationally prohibitive for operational regional, convective-scale deployment; moreover, the diffusion residual remains comparatively smooth on the sparse, heavy-tailed precipitation channel, where convective-scale extremes matter most.

This thesis builds on a two-stage StormCast model trained on a Taiwan-domain RWRF dataset — a frozen deterministic regression mean plus a generative residual head — and proposes and evaluates **FlowCast**: a residual generator based on Conditional Flow Matching (CFM) that is teacher-free and trained in a single stage. FlowCast regresses a straight-line noise-to-data velocity field v_θ and, at inference, samples by integrating it with only a



handful of explicit Euler steps — a direct, single-training-run alternative to the roughly 25-NFE EDM diffusion model that we retain as the reference baseline. Because the Taiwan RWRF target contains only four high-resolution output channels (t_{2m} , u_{10} , v_{10} , q_{pepre}), we can afford channel-specific loss design: a channel-weighted L2 objective plus a radial log power-spectral-density (log-PSD) regularizer on the sparse, heavy-tailed precipitation channel q_{pepre} . We additionally study the $\log_{1p}$ versus raw-mm/h precipitation encoding and the precipitation channel-weight w_{pr} .

We compare FlowCast head-to-head against the EDM diffusion baseline at a matched training-sample budget on the full 2022 hourly validation year, using a precipitation-aware metric suite: per-channel RMSE/MAE, probabilistic CRPS, categorical precipitation scores (CSI, FSS, HSS, false-alarm ratio) at operational thresholds, radial log-PSD, and autoregressive rollout error out to 12 hours. Our experiments show that, at roughly one-third of the inference wall-clock time, FlowCast matches or beats EDM diffusion on CRPS, wind-field RMSE, precipitation RMSE, and false-alarm ratio, and offers a better NFE/time-versus-quality trade-off. The thesis contributes a reproducible single-run flow-matching pipeline and an evaluation protocol for regional convective-scale generative forecasting.

Keywords: Flow Matching, Conditional Flow Matching, Diffusion Models, Generative Weather Forecasting, Convective-Scale Forecasting, Precipitation Nowcasting, StormCast







Contents

	Page
Verification Letter from the Oral Examination Committee	i
Acknowledgements	iii
摘要	v
Abstract	vii
Contents	xi
List of Figures	xvii
List of Tables	xix
Denotation	xxi
Chapter 1 Introduction	1
1.1 Weather Forecasting and Its Societal Stakes	1
1.2 A Brief History of Weather Prediction Methods	2
1.3 Diffusion Models for Weather: Intuition	4
1.4 The Inference-Cost Problem	4
1.5 From Diffusion to Flow Matching	5
1.6 Contributions	6
1.7 Thesis Outline	8



Chapter 2 Background and Related Work

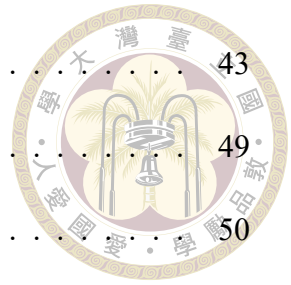
2.1	Numerical Weather Prediction	10
2.2	Data-Driven Global Weather Models	10
2.3	Regional and Convective-Scale Forecasting	11
2.4	GenCast and the Generative Turn	13
2.5	Score-Based Diffusion: A Primer	13
2.5.1	DDPM: The Discrete-Time Variance-Preserving Construction	14
2.5.2	DDIM: Non-Markovian Forward and Deterministic Sampling	16
2.5.3	Continuous-Time View: VE-SDE and PF-ODE	17
2.6	EDM: The Elucidated Formulation	18
2.7	Conditioning in Diffusion Models	20
2.8	Flow Matching and FlowCast	21
2.9	Flow Matching versus Diffusion: Why Straight-Line Paths Help . . .	23
2.10	Prior Generative Models for Precipitation Nowcasting	25
2.11	Evaluation of Precipitation Forecasts	26

Chapter 3 Method

29

3.1	Problem Setup	29
3.2	Dataset: Taiwan RWRP	31
3.3	Data Sources and Preprocessing Pipeline	32
3.3.1	Raw data sources	33
3.3.2	Preprocessing pipeline	34
3.3.3	Target Distributions: Sparsity, Gaussianity, and the Diffusion Prior .	39
3.4	Regression Mean and the EDM Diffusion Baseline	42

3.5	The FlowCast Residual (Conditional Flow Matching)	43
3.6	Loss Design for the Precipitation Channel	49
3.7	Training Infrastructure	50
Chapter 4 Experiments and Results		53
4.1	Experimental Protocol	53
4.2	Evaluation Metrics	55
4.3	FlowCast versus Diffusion: Headline Comparison	56
4.4	Inference Cost: NFE and Wall-Clock Trade-off	59
4.5	Precipitation Skill by Threshold	60
4.6	Autoregressive Rollout	61
4.7	Ablation: $\log_{1p}$ Precipitation Encoding	62
4.8	Ablation: Precipitation Channel Weight	63
4.9	Comparison Against the Legacy Baseline	64
4.10	Qualitative Case Studies	65
4.11	Failure Modes	68
4.12	Discussion	68
Chapter 5 Conclusion and Future Work		71
5.1	Summary of Contributions	71
5.2	Limitations	72
5.3	Future Work	73
5.4	Closing Remarks	74



References

Appendix A — Hyperparameters and Training Configuration

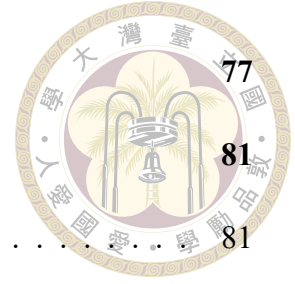
A.1	EDM Diffusion Baseline Hyperparameters	81
A.2	FlowCast (Conditional Flow Matching)	82
A.3	Ablation Grids	84
A.4	Per-Channel Dataset Statistics	84

Appendix B — Reproducibility

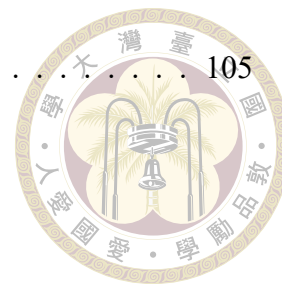
B.1	Code Availability	85
B.2	Dataset and Checkpoints	86
B.3	Commands	86

Appendix C — Detailed Derivations

C.1	DDPM Forward-Process Marginals	89
C.2	DDPM Posterior $q(x_{t-1} x_t, x_0)$	91
C.3	DDPM Variational Bound (ELBO)	92
C.4	DDPM Simplified ϵ -Loss	93
C.5	Tweedie’s Formula: Score and Denoiser	95
C.6	Probability-Flow ODE from the VE-SDE	95
C.7	DDIM: Non-Markovian Forward and Deterministic Sampler	97
C.8	Equivalence of ϵ , x_0 , and v Parameterisations	98
C.9	EDM Preconditioning Derivation	99
C.10	Karras ρ -Spaced Noise Schedule	102
C.11	CFM Theorem: Conditional = Marginal Gradient	102
C.12	Independent CFM: Straight-Line Path and Target	103



C.13	FlowCast’s Standardised-Residual Adaptation	105
------	---	-----







List of Figures

Figure 3.1	Per-channel Gaussianity diagnostics for HighRes targets	39
Figure 4.1	FlowCast versus diffusion scoreboard plot	57
Figure 4.2	Per-channel RMSE and CRPS	59
Figure 4.3	Categorical precipitation skill by threshold	60
Figure 4.4	Heavy-rain Fractions Skill Score	61
Figure 4.5	Rollout RMSE versus lead time	61
Figure 4.6	Qualitative precipitation panels	66
Figure 4.7	Qualitative wind panel	67
Figure 4.8	FlowCast training curves and spectrum	67





List of Tables

Table 3.1	Raw data sources used for training. Low-resolution ERA5 provides synoptic conditioning; RWRF and QPEPRE together supply the convection-permitting target.	34
Table 3.2	Gaussianity statistics of HighRes target channels	40
Table 4.1	FlowCast versus EDM diffusion scoreboard	57
Table 4.2	log1p versus raw-mm/h encoding ablation	62
Table 4.3	Precipitation channel-weight ablation	64
Table 4.4	Legacy versus cleaned pipeline	65





Denotation

NWP	Numerical Weather Prediction
RWRF	Regional Weather Research and Forecasting (Taiwan domain reanalysis)
EDM	Elucidated Diffusion Model (Karras et al., 2022)
CFM	Conditional Flow Matching
I-CFM	Independent Conditional Flow Matching
ODE	Ordinary Differential Equation
PF-ODE	Probability-Flow Ordinary Differential Equation
SDE	Stochastic Differential Equation
NFE	Number of Function Evaluations (neural-net forward passes per sample)
EMA	Exponential Moving Average
PSD	Power Spectral Density

CSI	Critical Success Index
FSS	Fractions Skill Score
HSS	Heidke Skill Score
FAR	False-Alarm Ratio
CRPS	Continuous Ranked Probability Score
X_t	High-resolution atmospheric state at time t (target). Channels: τ_{2m} , u_{10} , v_{10} , q_{pepre} .
S_t	Low-resolution conditioning state at time t (24 channels).
I	Time-invariant fields (land–sea mask, orography).
F_ξ	Deterministic regression network producing mean $M_t = F_\xi(X_{t-1}, S_t, I)$.
M_t	Regression mean prediction.
R_t	Residual $R_t = X_t - M_t$ learned by the generative head.
c	Conditioning bundle (EDM: $\text{concat}(X_{t-1}, S_t, M_t, I)$; FlowCast: $\text{concat}(X_{t-1}, M_t, I)$).
D_ψ	EDM diffusion baseline denoiser (EDMPrecond SongUNet).
v_θ	FlowCast velocity field (FlowCastPrecond SongUNet).
σ	EDM noise level.
$\sigma_{\min}, \sigma_{\max}$	Minimum and maximum EDM noise levels (0.002 and 80).



σ_{data}

Data standard-deviation hyperparameter shared by EDM and Flow-Cast (0.5).

ρ

Karras schedule exponent (7).

t

FlowCast flow time, $t \in [0, 1]$ (noise at $t = 0$, data at $t = 1$).

x_0, x_1

Flow endpoints: prior sample $x_0 \sim \mathcal{N}(0, I)$ and standardised residual $x_1 = R_t / \sigma_{\text{data}}$.

u_t

Target velocity field of Independent CFM, $u_t = x_1 - x_0$.

σ_{path}

I-CFM conditional-path standard deviation (0.01).

K

Number of ODE sampling steps (Euler), equal to the NFE of Flow-Cast.

β_k

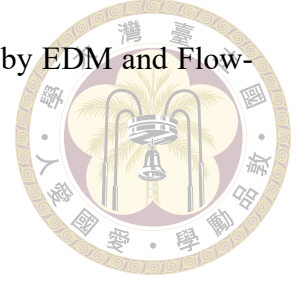
Per-channel loss weight; $w_{\text{pr}} = \beta_{\text{qpepre}}$ is the precipitation weight.

λ_{spec}

Weight of the radial log-PSD spectral regulariser.

$P(\kappa)$

Radial power spectral density at wavenumber κ .







Chapter 1 Introduction

1.1 Weather Forecasting and Its Societal Stakes

Accurate short-range weather forecasting is a public-safety service. In Taiwan, a small, densely populated island lying along the western Pacific typhoon track, the stakes are especially concentrated: a single landfalling typhoon can dump more than 1000 mm of rainfall on steep orography in twelve hours, trigger flash floods and landslides, and disrupt transportation, agriculture, and power. Between typhoons, Mei-yu frontal systems, afternoon convective storms, and orographically forced rainfall drive the remainder of the hydrological cycle. For decision-makers who must issue evacuation orders, schedule reservoir releases, or re-route air traffic, the relevant question is almost always probabilistic and local: how much rain, where, within the next six to twelve hours?

The operationally critical quantity — precipitation at convective scales — is also the hardest for weather models to predict. Precipitation fields are sparse (most pixels are dry), heavy-tailed (the few wet pixels span orders of magnitude), and strongly coupled to terrain and mesoscale dynamics that only emerge at kilometre-scale resolution. This combination means that any forecasting system which produces visually smooth outputs, or which regresses toward the mean, will systematically underpredict the very events that matter most.

1.2 A Brief History of Weather Prediction Methods



Modern weather forecasting has passed through four broad eras, each driven by a different scientific or computational advance.

Numerical Weather Prediction (NWP). Since Richardson's early twentieth-century vision and its post-war realisation by Charney, Phillips, and others, operational forecasting has been dominated by numerical integration of the primitive equations of atmospheric motion. State-of-the-art systems such as the ECMWF Integrated Forecasting System (IFS), the NCEP Global Forecast System (GFS), and regional convection-permitting models like WRF and its Taiwan variant RWRF discretise the atmosphere on a three-dimensional grid, solve the governing fluid-dynamics and thermodynamics equations, and close the unresolved sub-grid processes (convection, microphysics, boundary-layer turbulence, radiation) through physical parameterisations. Uncertainty is quantified by ensemble prediction systems that perturb initial conditions and model physics. NWP is physically consistent, well-understood, and remarkably skilful, but convection-permitting runs are computationally expensive, parameterisation error is non-trivial, and operational latency is a fundamental constraint.

Statistical and classical machine-learning post-processing. Long before deep learning, statistical techniques — model output statistics (MOS), analog methods, kriging, and more recently random forests and gradient-boosted trees — have been used to bias-correct and downscale NWP outputs. These approaches treat the NWP forecast as a feature vector and learn a local correction to each observation site. They set an important precedent:

that learned components can complement physical cores, and that the ultimate criterion is predictive skill at points of interest rather than global physical fidelity.



The deep-learning era. Starting around 2020, a new generation of purely data-driven global weather models began to rival and in some cases surpass operational NWP at synoptic scales. FourCastNet [13] used an Adaptive Fourier Neural Operator to produce six-hourly global forecasts at 0.25° resolution. Pangu-Weather [2] introduced a three-dimensional hierarchical transformer and demonstrated medium-range skill competitive with IFS. GraphCast [9] adopted an icosahedral-mesh graph neural network and achieved operational-quality ten-day forecasts at a fraction of the inference cost of IFS. At convective scales, MetNet [1, 4, 20] and the Deep Generative Model of Rainfall (DGMR) [15] pushed radar-based nowcasting from deterministic CNNs toward generative models capable of producing sharp, ensemble-like outputs. Most recently, StormCast [12] demonstrated that a convection-permitting diffusion model can learn a km-scale regional forecaster directly from reanalysis data.

The generative turn. A persistent problem of deterministic deep-learning forecasters is that, because they are trained with pixelwise regression losses, they produce fields that are systematically blurry and that underestimate extremes. Generative models resolve this by learning the distribution of plausible forecasts rather than its conditional mean. DGMR showed this qualitatively for radar nowcasting; GenCast [14] made the argument at global scale, demonstrating that a conditional diffusion model produces both sharper and better-calibrated ensemble forecasts than GraphCast. The lesson transfers directly to regional convective forecasting, and it motivates the generative forecaster at the centre of this thesis.

1.3 Diffusion Models for Weather: Intuition



A diffusion model is a generative model trained to reverse a gradual noising process.

Given a clean sample x_0 drawn from some data distribution — say, a high-resolution precipitation field over Taiwan — we progressively add Gaussian noise until after enough steps the result is indistinguishable from pure noise. A neural network is then trained to undo one step of this process: given a noised sample and the current noise level, predict the underlying clean signal (equivalently, the score of the noised distribution). At sampling time we start from pure Gaussian noise and apply the learned denoiser repeatedly until we recover a clean sample.

Three properties make this framework attractive for weather forecasting. First, it is naturally probabilistic: different noise seeds produce different but equally plausible samples, giving an ensemble for free. Second, the samples are sharp: because the model is never asked to average over outcomes, it does not regress toward the mean. Third, the denoiser can be conditioned on auxiliary inputs — the previous atmospheric state, coarse boundary fields, invariant land-surface information — which is exactly the interface a forecaster needs. GenCast at global scale and StormCast at regional scale both exploit these properties.

1.4 The Inference-Cost Problem

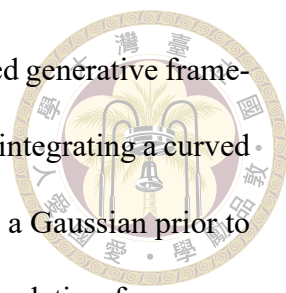
The cost of this flexibility is paid at inference time. High-quality diffusion sampling requires integrating the model's probability-flow ordinary differential equation (PF-ODE) with on the order of 25 neural-network forward passes per generated sample — the so-

called number of function evaluations (NFE). For a single forecast step this is tolerable, but operational use cases compound it. An autoregressive 24-hour rollout multiplies the cost by 24. A 50-member ensemble for probabilistic forecasting multiplies it again. A convection-permitting Taiwan-domain forecast with a 50-member ensemble over a 24-hour horizon therefore requires on the order of $50 \times 24 \times 25 = 30,000$ forward passes of a large U-Net — which places the entire class of generative forecasters near the edge of, or outside, the operational envelope of a regional forecasting centre that must refresh its nowcast every few minutes between radar volumes.

The cost is not the only issue. The reason diffusion needs so many steps is that its PF-ODE trajectory — the path that carries a pure-noise sample to a clean forecast — is curved. A curved trajectory cannot be integrated accurately in one or two steps; coarse step counts incur visible discretisation error, and that error is worst precisely on the sparse, heavy-tailed precipitation channel, where the score of the noised distribution is ill-conditioned near small noise levels. The practical symptom is that an aggressively sub-sampled diffusion residual reverts toward a smooth, “slightly wet everywhere” field that scores acceptably on RMSE but destroys the convective structure operational users care about.

1.5 From Diffusion to Flow Matching

Both problems — the step count and the curvature — point at the same lever: the shape of the generative trajectory. If the path from noise to data were a straight line rather than a curved diffusion ODE, then a handful of explicit Euler steps would integrate it almost exactly, and a single step would already be a strong approximation.

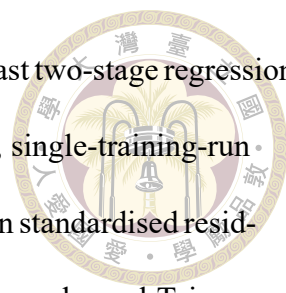


Conditional Flow Matching (CFM) [10, 24] is a recently-proposed generative framework that learns exactly such a model. Instead of learning a score and integrating a curved PF-ODE, CFM regresses a deterministic velocity field that transports a Gaussian prior to the data along a straight-line probability path, and it does so with a simulation-free mean-squared-error objective that needs no teacher model at all. (author?) [16] applied this idea to radar precipitation nowcasting under the name **FlowCast** and reported that a 1-to-10-step flow-matching sampler matched or beat a 100-step diffusion sampler on probabilistic precipitation skill at a fraction of the inference cost.

This thesis adapts FlowCast to the regional convection-permitting StormCast setting on the Taiwan RWRf domain. We keep StormCast’s two-stage decomposition — a frozen deterministic regression mean plus a generative residual head — and replace the residual head’s curved EDM diffusion process with a straight-line conditional-flow-matching velocity field. The EDM diffusion residual is retained throughout as the reference baseline against which FlowCast is measured at a matched training-sample budget, so that every quality and cost difference can be attributed to the choice of generative trajectory rather than to data, architecture, or the regression mean, all of which are held fixed. FlowCast is not a distillation of the diffusion teacher: it is trained from scratch in a single run, which makes it both simpler to train and a clean test of whether a straight-line prior alone — without any teacher signal — already delivers the speed and sharpness that operational regional forecasting needs.

1.6 Contributions

This thesis makes the following contributions:

- 
1. We adapt Conditional Flow Matching [10, 16, 24] to the StormCast two-stage regression-plus-residual architecture, producing **FlowCast**, a teacher-free, single-training-run residual forecaster that learns a straight-line noise-to-data ODE in standardised residual space and samples it with a few explicit Euler steps on a four-channel Taiwan regional target.
 2. We carry out the engineering needed to make the flow-matching residual share exactly the frozen regression mean and per-channel normalisation of the EDM diffusion baseline, so that a head-to-head comparison isolates the generative trajectory: same data, same F_{ξ} , same σ_{data} , same backbone, only the EDM process versus the CFM velocity field.
 3. We exploit the four-channel target with a precipitation-aware loss: a channel-weighted L2 objective plus an auxiliary radial power-spectral-density (log-PSD) regulariser on the sparse, heavy-tailed qpepre channel, and we ablate the precipitation channel weight w_{pr} and the log1p-versus-raw-mm/h precipitation encoding.
 4. We assemble a precipitation-aware evaluation suite — per-channel RMSE/MAE, probabilistic CRPS, categorical skill scores (CSI, FSS, HSS, false-alarm ratio) at operational thresholds, radial log-PSD, and 1–12-hour rollout errors — and run it on the full 2022 hourly validation year of the Taiwan RWRF dataset.
 5. We present a head-to-head comparison of FlowCast against an EDM diffusion residual at a matched sample budget, including an NFE/wall-clock-versus-quality trade-off, and against a legacy old-StormCast baseline, on a regional convection-permitting forecaster — to our knowledge the first such study of flow matching on the Taiwan convective-scale domain.

6. Appendix B.3 provides full line-by-line derivations of DDPM, DDIM, VE-SDE / PF-ODE, EDM preconditioning, and Conditional Flow Matching — the theoretical machinery both methods in this thesis rest on — in a form that should let a reader with a working knowledge of Gaussian algebra and the Fokker–Planck equation reconstruct every identity from scratch.

1.7 Thesis Outline

Chapter 2 surveys weather-prediction methods, diffusion theory (the basis of the EDM baseline), and conditional flow matching (the basis of FlowCast) in enough depth to motivate and define every component used later. Chapter 3 describes the dataset, the frozen regression mean and EDM diffusion baseline, and the FlowCast pipeline as it is implemented in the accompanying codebase. Chapter 4 presents the experimental protocol and the FlowCast-versus-diffusion results across NFE budgets, precipitation thresholds, and rollout horizons. Chapter 5 summarises findings, limitations, and future work.



Chapter 2 Background and Related Work

This chapter assembles the theoretical background needed to understand the remainder of the thesis. It is organised in four parts: a survey of weather-prediction methods from NWP to data-driven models (Sections 2.1–2.4); the theory of score-based diffusion models, starting from the discrete-time DDPM construction, proceeding through DDIM and the continuous variance-exploding SDE / probability-flow ODE, and culminating in the EDM formulation used by our diffusion baseline (Sections 2.5–2.7); the theory of Conditional Flow Matching and FlowCast — the straight-line, teacher-free generative model at the centre of this thesis — together with a direct comparison against the curved diffusion path and a survey of prior generative precipitation models (Sections 2.8–2.10); and an overview of precipitation-aware evaluation metrics (Section 2.11). Throughout the chapter we state the key identities in the main text and defer line-by-line derivations to Appendix B.3, which the reader may consult for any step marked “(see Appendix B.3)”.

2.1 Numerical Weather Prediction



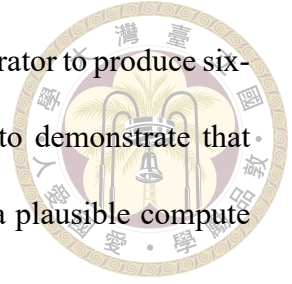
Numerical weather prediction discretises the primitive equations of atmospheric motion — continuity, momentum (Navier–Stokes on a rotating sphere), thermodynamic energy, and water-vapour conservation — onto a three-dimensional grid and advances the state forward in time via explicit or semi-implicit integration. Sub-grid processes not captured by the dynamical core (convection, microphysics, boundary-layer turbulence, radiation, land-surface exchange) are closed through physical parameterisations. Operational systems include ECMWF’s IFS (global spectral, 0.1° horizontal resolution), NCEP’s GFS, and regional limited-area models such as WRF and its Taiwan-tuned variant RWRF, which provide the high-resolution targets used in this thesis.

Uncertainty is quantified by ensemble prediction systems that perturb initial conditions (e.g. via singular-vector or ensemble Kalman methods) and model physics. A convection-permitting (sub-4 km) ensemble with a few tens of members and a 24-hour forecast horizon requires hours of dedicated supercomputer time. The practical consequence is that NWP ensemble size and spatial resolution are traded off against operational latency.

2.2 Data-Driven Global Weather Models

Starting in 2022, a series of deep-learning models demonstrated that a single neural network, trained on the ERA5 reanalysis, can produce global medium-range forecasts that rival operational NWP on many headline metrics at a fraction of the inference cost.

FourCastNet. (author?) [13] used an Adaptive Fourier Neural Operator to produce six-hourly global forecasts at 0.25° resolution. It was the first model to demonstrate that a data-driven system could match IFS on synoptic variables within a plausible compute budget.



Pangu-Weather. (author?) [2] introduced a hierarchical three-dimensional transformer with a hierarchical time-aggregation scheme and demonstrated medium-range skill competitive with or exceeding IFS on several variables and lead times.

GraphCast. (author?) [9] adopted a multi-mesh graph neural network defined on an icosahedral discretisation of the sphere. It produces ten-day 0.25° forecasts that are competitive with IFS on the majority of evaluated fields and does so in roughly one minute on a single TPU, compared with hours on a supercomputer for NWP.

These models share an important limitation: because they are trained with deterministic pixelwise losses (typically latitude-weighted MSE), their forecasts become progressively blurrier with lead time and systematically underestimate extreme events. For convection and precipitation, which this thesis cares about most, this is disqualifying.

2.3 Regional and Convective-Scale Forecasting

Global models operate at resolutions too coarse to resolve convection and orographic rainfall over Taiwan. Three lines of work fill the gap.

Nowcasting models. MetNet-1/2/3 [1, 4, 20] produce high-resolution short-range precipitation forecasts by ingesting ground-based radar and satellite observations through

large convolutional/attention backbones. DGMR [15] uses a conditional GAN to produce sharp, ensemble-like radar nowcasts and was the first work to make a strong empirical case that generative modelling is essential for precipitation skill.



StormCast. (author?) [12] introduced a convection-permitting regional diffusion forecaster trained on the High-Resolution Rapid Refresh (HRRR) reanalysis over CONUS. StormCast decomposes forecasting into two stages:

1. A deterministic regression network F_ξ (a StormCastUNet) takes the previous high-resolution state X_{t-1} , a low-resolution conditioning state S_t , and time-invariant fields I , and predicts the conditional mean $M_t = F_\xi(X_{t-1}, S_t, I)$.
2. A residual diffusion network D_ψ (an EDMPrecond SongUNet) models the residual $R_t = X_t - M_t$ conditionally on $c = \text{concat}(X_{t-1}, S_t, M_t, I)$.

At inference time the model is rolled out autoregressively, with the previous diffusion output feeding the next step's regression input. This decomposition has two payoffs: the deterministic network captures what is easy to predict with a single network call, leaving the diffusion model free to focus on the residual uncertainty; and the residual is closer to Gaussian than the raw fields, which stabilises diffusion training.

This thesis uses a Taiwan-adapted StormCast model trained on the RWRF dataset. The adaptation reduces the output state from the 99 channels of the CONUS original to just four: two-metre temperature (t2m), ten-metre zonal and meridional winds (u10, v10), and hourly precipitation (qpepre). This channel reduction has a useful side-effect: with only four output channels, per-channel loss engineering becomes cheap, and we exploit this in the precipitation-aware losses of Chapter 3.

2.4 GenCast and the Generative Turn

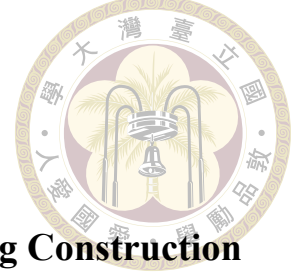


(author?) [14] introduced GenCast, a global 0.25° conditional diffusion forecaster that samples ensemble members by varying the initial noise seed. GenCast was the first diffusion forecaster to clearly surpass the leading deterministic model (GraphCast) across the majority of its evaluation suite, on both deterministic scores (RMSE) and probabilistic scores (CRPS). Its success is the strongest empirical argument for the claim that motivates this thesis: generative modelling — and diffusion in particular — is the right inductive bias for short-to-medium range weather forecasting, but its operational cost is the obstacle to practical deployment.

2.5 Score-Based Diffusion: A Primer

A diffusion model defines a forward stochastic process that progressively corrupts a data sample $x_0 \sim p_{\text{data}}$ with Gaussian noise until the terminal state is indistinguishable from pure noise, then learns a neural network that reverses the corruption. The apparent difficulty of this programme — fitting the full generative reverse process of a high-dimensional distribution — hides a sequence of algebraic identities that together reduce training to a mean-squared-error regression. This section traces those identities in the order in which they were historically discovered: first the discrete-time variance-preserving DDPM construction of (author?) [6]; then the non-Markovian generalisation and deterministic sampler of DDIM [21]; then the continuous variance-exploding SDE and the probability-flow ODE that unify both [23]; and finally the elucidated EDM parameterisation used by our diffusion baseline [8]. Detailed line-by-line derivations of every equation

in this section are collected in Appendix B.3.



2.5.1 DDPM: The Discrete-Time Variance-Preserving Construction

Forward process. DDPM defines a T -step Markov chain that adds Gaussian noise according to a variance schedule $\{\beta_t\}_{t=1}^T$:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I). \quad (2.1)$$

Setting $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, induction (Appendix B.3, §C.1) gives the closed-form marginal

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I), \quad x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (2.2)$$

This is the first key tractability result: the forward process never has to be simulated. For any training timestep t the corrupted sample x_t is drawn in one line using x_0 and a fresh Gaussian ϵ .

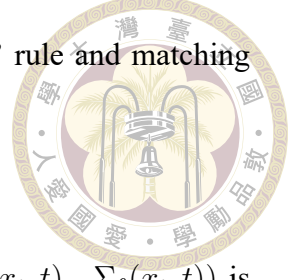
Tractable posterior. The reverse step $q(x_{t-1} | x_t)$ is intractable without x_0 , but the posterior conditioned on x_0 is Gaussian in closed form:

$$q(x_{t-1} | x_t, x_0) = \mathcal{N}(\tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I), \quad (2.3)$$

with

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\bar{\alpha}_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t, \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t. \quad (2.4)$$

The full derivation — combining Equations (2.1)–(2.2) with Bayes’ rule and matching coefficients of a quadratic in x_{t-1} — is given in Appendix B.3, §C.2.



Variational bound. The reverse process $p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$ is trained to match q . The evidence lower bound (ELBO) on $\log p_\theta(x_0)$ decomposes into a sum of per-step KL divergences

$$-\log p_\theta(x_0) \leq \underbrace{D_{\text{KL}}(q(x_T \mid x_0) \parallel p(x_T))}_{L_T} + \sum_{t=2}^T \underbrace{D_{\text{KL}}(q(x_{t-1} \mid x_t, x_0) \parallel p_\theta(x_{t-1} \mid x_t))}_{L_{t-1}} - \underbrace{\log p_\theta(x_0 \mid x_1)}_{L_0}, \quad (2.5)$$

derived from Jensen’s inequality on the joint $q(x_{1:T} \mid x_0)$ (Appendix B.3, §C.3). The noise-schedule design ensures L_T is approximately $D_{\text{KL}}(\mathcal{N}(0, I) \parallel \mathcal{N}(0, I)) = 0$ for large T ; L_0 is a discretised log-likelihood treated separately; the bulk of the loss is the middle sum, each term of which is a KL between two Gaussians with known variances.

Simplification to MSE. Because both sides are Gaussians with matched variances $\Sigma_\theta = \tilde{\beta}_t I$, the KL collapses to the squared distance of means:

$$L_{t-1} = \frac{1}{2\tilde{\beta}_t} \mathbb{E}_{x_0, \epsilon} [\|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2] + \text{const.} \quad (2.6)$$

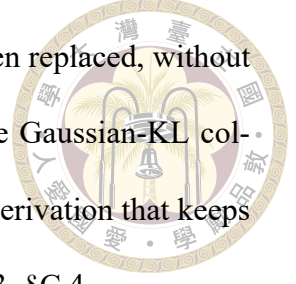
Reparameterising μ_θ through a noise predictor ϵ_θ ,

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right), \quad (2.7)$$

and dropping the σ -dependent prefactor (empirically stable), yields the DDPM training loss used in practice:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2]. \quad (2.8)$$

Equation (2.8) is the punchline: the intractable log-likelihood has been replaced, without approximation on its upper bound and without approximation in the Gaussian-KL collapse, by a supervised regression of the added noise. A line-by-line derivation that keeps every constant and justifies the dropped weighting is in Appendix B.3, §C.4.



2.5.2 DDIM: Non-Markovian Forward and Deterministic Sampling

DDPM’s sampler draws $T = 1000$ stochastic reverse steps, which is too slow for deployment. (author?) [21] observed that the DDPM training loss (2.8) depends on the marginals $q(x_t \mid x_0)$ but not on the particular Markov chain used to produce them. Any non-Markovian forward process with the same marginals yields the same trained network, but admits a different — potentially deterministic — sampler. DDIM parameterises such a family by

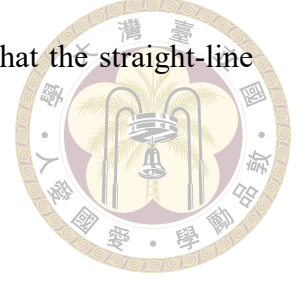
$$q_\sigma(x_{t-1} \mid x_t, x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_{t-1}} x_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \frac{x_t - \sqrt{\bar{\alpha}_t} x_0}{\sqrt{1 - \bar{\alpha}_t}}, \sigma_t^2 I\right), \quad (2.9)$$

parameterised by a free noise scale $\sigma_t \in [0, \tilde{\beta}_t]$. Setting $\sigma_t = 0$ yields a deterministic update

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0(x_t, t) + \sqrt{1 - \bar{\alpha}_{t-1}} \frac{x_t - \sqrt{\bar{\alpha}_t} \hat{x}_0(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}, \quad (2.10)$$

where $\hat{x}_0(x_t, t) = (x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)) / \sqrt{\bar{\alpha}_t}$ is the implied clean-data prediction. Equation (2.10) defines an integrator of an ODE: the same network, trained once, can be sampled with arbitrary sub-schedules and in far fewer steps (typically 20–50 NFE with comparable quality). A derivation in the equivalent variance-exploding form appears in Appendix B.3, §C.7. DDIM is the simplest first-order integrator of the underlying PF-ODE; the persistence of curvature in that ODE — which is what forces the step count

to stay in the 20–50 range for sharp precipitation — is precisely what the straight-line flow-matching path of Section 2.8 is designed to remove.



2.5.3 Continuous-Time View: VE-SDE and PF-ODE

(author?) [23] unified DDPM, noise-conditional score matching, and their continuous-time limits into a single SDE framework. In the variance-exploding (VE) formulation adopted by EDM and inherited by StormCast, the forward SDE is

$$dx = \sqrt{\frac{d\sigma^2(t)}{dt}} dW_t, \quad (2.11)$$

so that the closed-form marginal is the same one-line Gaussian corruption

$$x_\sigma = x_0 + \sigma \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad (2.12)$$

now indexed by a continuous noise level $\sigma \in [0, \sigma_{\max}]$. The remarkable structural fact is that every diffusion SDE admits a deterministic probability-flow ordinary differential equation (PF-ODE) whose marginals coincide with those of the SDE at every time:

$$\frac{dx}{d\sigma} = -\sigma \nabla_x \log p_\sigma(x), \quad (2.13)$$

where $\nabla_x \log p_\sigma(x)$ is the (Stein) score of the noised marginal. Equation (2.13) is derived in Appendix B.3, §C.6 by matching the Fokker–Planck equation of the SDE against the continuity equation of the deterministic flow. The PF-ODE plays a central role here: it defines a deterministic trajectory $\sigma \mapsto x(\sigma)$ that links a pure-noise sample $x(\sigma_{\max})$ to its corresponding clean sample $x(\sigma_{\min})$, and any method that shortens the number of sampling steps — a few-step diffusion sampler, or the straight-line flow of Section 2.8 — is, one

way or another, trying to traverse this kind of trajectory with fewer function evaluations.



Score–denoiser equivalence (Tweedie). Training a diffusion model amounts to learning the score, typically indirectly through a denoiser $D(x_\sigma; \sigma)$ whose output predicts $\mathbb{E}[x_0 \mid x_\sigma]$. The score and the denoiser are related by Tweedie’s formula,

$$\nabla_x \log p_\sigma(x) = \frac{D(x; \sigma) - x}{\sigma^2}, \quad (2.14)$$

which we derive from the MMSE-denoiser identity in Appendix B.3, §C.5. The practical consequence is that denoising-score matching reduces to an ℓ_2 regression of the clean data from its noised version:

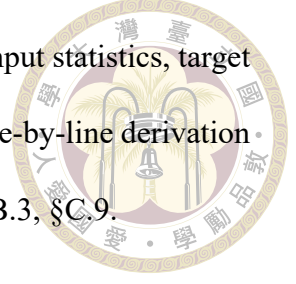
$$\mathcal{L}_{\text{DSM}}(\sigma) = \mathbb{E}_{x_0, \epsilon} [\|D_\theta(x_0 + \sigma\epsilon; \sigma) - x_0\|^2]. \quad (2.15)$$

Equation (2.15) is the continuous-time counterpart of Equation (2.8): the tractable training objective is a weighted mean-squared error over the noise scale, and the network can equivalently predict the clean data x_0 , the noise ϵ , or the $v = \alpha_t \epsilon - \sigma_t x_0$ target of (author?) [18]. The three parameterisations are interconvertible linearly and differ only in which pair of quantities the network sees as inputs and outputs; see Appendix B.3, §C.8.

2.6 EDM: The Elucidated Formulation

(author?) [8] proposed a unified parameterisation of diffusion training and sampling that cleans up a number of ad-hoc choices in earlier DDPM-style models. The EDM diffusion baseline of this thesis uses this formulation in full, and FlowCast borrows its data scale σ_{data} from it (Chapter 3). The EDM preconditioning scalings are not heuristics:

they are the unique one-parameter family that makes the denoiser's input statistics, target statistics, and per- σ loss scale simultaneously uniform across σ . A line-by-line derivation of the $c_{\text{skip}}, c_{\text{out}}, c_{\text{in}}, c_{\text{noise}}, \lambda(\sigma)$ formulas below is given in Appendix B.3, §C.9.



Noise schedule. EDM uses a ρ -spaced discretisation of noise levels between $\sigma_{\min}$ and $\sigma_{\max}$:

$$\sigma_i = \left(\sigma_{\max}^{1/\rho} + \frac{i}{N-1} (\sigma_{\min}^{1/\rho} - \sigma_{\max}^{1/\rho}) \right)^\rho, \quad (2.16)$$

for $i = 0, \dots, N-1$, with $\sigma_{\min} = 0.002$, $\sigma_{\max} = 80$, and $\rho = 7$. The schedule concentrates noise levels near $\sigma_{\min}$, which is where most of the useful signal is generated.

Preconditioning. A naive denoiser $F_\theta(x_\sigma, \sigma)$ must cope with inputs whose magnitudes vary by orders of magnitude. EDM addresses this by wrapping the inner network in skip, input, output, and noise scalings:

$$D_\theta(x_\sigma; \sigma) = c_{\text{skip}}(\sigma) x_\sigma + c_{\text{out}}(\sigma) F_\theta(c_{\text{in}}(\sigma) x_\sigma, c_{\text{noise}}(\sigma)), \quad (2.17)$$

where

$$\begin{aligned} c_{\text{skip}}(\sigma) &= \sigma_{\text{data}}^2 / (\sigma^2 + \sigma_{\text{data}}^2), & c_{\text{out}}(\sigma) &= \sigma \cdot \sigma_{\text{data}} / \sqrt{\sigma^2 + \sigma_{\text{data}}^2}, \\ c_{\text{in}}(\sigma) &= 1 / \sqrt{\sigma^2 + \sigma_{\text{data}}^2}, & c_{\text{noise}}(\sigma) &= \frac{1}{4} \log \sigma. \end{aligned}$$

These scalings ensure that F_θ always sees inputs of unit variance and predicts an unnormalised residual, which is numerically stable across the full range of σ .

Training loss. EDM trains with a weighted denoising-score-matching loss:

$$\mathcal{L}_{\text{EDM}} = \mathbb{E}_{x_0, \sigma, \epsilon} \left[\lambda(\sigma) \|D_\theta(x_0 + \sigma\epsilon; \sigma) - x_0\|^2 \right], \quad (2.18)$$



with a log-normal distribution over σ and a Karras-specified weighting $\lambda(\sigma)$.

Heun sampler. EDM samples via a second-order Heun integrator of the PF-ODE. Given x_i at noise level σ_i , the next sample is

$$d_i = (x_i - D_\theta(x_i; \sigma_i)) / \sigma_i, \quad (2.19)$$

$$x_{i+1}^{(\text{euler})} = x_i + (\sigma_{i+1} - \sigma_i) d_i, \quad (2.20)$$

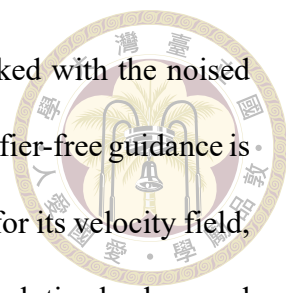
$$d'_i = (x_{i+1}^{(\text{euler})} - D_\theta(x_{i+1}^{(\text{euler})}; \sigma_{i+1})) / \sigma_{i+1}, \quad (2.21)$$

$$x_{i+1} = x_i + (\sigma_{i+1} - \sigma_i) \frac{1}{2} (d_i + d'_i). \quad (2.22)$$

Each step uses two denoiser evaluations except the final step where the correction is skipped. With $N = 18$ steps this yields roughly $2N - 1 = 35$ NFE; StormCast uses a shorter $N \approx 13$ schedule giving ≈ 25 NFE. This ≈ 25 -NFE Heun sampler is the cost figure that the straight-line FlowCast sampler of Chapter 3 is benchmarked against: every NFE that FlowCast saves at matched forecast quality is a direct operational speedup.

2.7 Conditioning in Diffusion Models

Conditional diffusion models can be implemented either by concatenating the condition c along the channel axis of the denoiser's input, or by classifier-free guidance (CFG) [7], which trains the network jointly on conditional and unconditional tasks and linearly combines their outputs at inference time. StormCast uses the simpler concatenation ap-



proach: the conditioning bundle $c = \text{concat}(X_{t-1}, S_t, M_t, I)$ is stacked with the noised target along the channel dimension and fed to the SongUNet. No classifier-free guidance is used. FlowCast inherits the same channel-concatenation mechanism for its velocity field, though it conditions on a slightly trimmed bundle that omits the low-resolution background S_t (Section 3.5); the fundamental change between the diffusion baseline and FlowCast is the generative process itself, not the way conditioning enters the network.

2.8 Flow Matching and FlowCast

Conditional Flow Matching (CFM) is a recently-proposed alternative to score-based diffusion that learns the same kind of noise-to-data generative model — a deterministic flow integrated from a simple prior to the data distribution — but with a training objective that bypasses score matching entirely. CFM is the generative method at the centre of this thesis (the FlowCast residual of Section 3.5), so we give it the same theoretical treatment as DDPM, DDIM, and EDM.

Continuous normalising flows, simulation-free training. A continuous normalising flow (CNF; 3) parameterises a deterministic velocity field $v_\theta(x, t)$, $t \in [0, 1]$, and pushes a prior sample $x_0 \sim \mathcal{N}(0, I)$ along the ODE $\dot{x}_t = v_\theta(x_t, t)$ to obtain x_1 , which we would like to distribute as p_{data} . Classical maximum-likelihood CNF training requires evaluating $\log p_\theta(x_1)$ via the Jacobian trace over the whole trajectory — a prohibitive cost at image or video scale. (author?) [10] and (author?) [24] instead train v_θ by direct regression against a target vector field $u_t(x \mid z)$ associated with a tractable conditional probability

path $p_t(x | z)$ indexed by a latent z :

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t \sim \mathcal{U}(0,1), z \sim q(z), x \sim p_t(x|z)} \left[\|v_\theta(x, t) - u_t(x | z)\|^2 \right] \quad (2.23)$$



The key theoretical result [10, Thm. 2] is that the gradient of (2.23) with respect to θ equals the gradient of the marginal flow-matching loss with the intractable marginal target field — so training the network against a conditional target yields an unbiased estimator of the true field. A derivation of this identity, along with the constants that are dropped because they do not depend on θ , is given in Appendix B.3, §C.11.

Independent CFM (I-CFM) and the straight-line path. The simplest instantiation, Independent CFM [24], takes $z = (x_0, x_1)$ with $x_0 \sim \mathcal{N}(0, I)$ independent of $x_1 \sim p_{\text{data}}$, and defines the straight-line Gaussian probability path

$$p_t(x | x_0, x_1) = \mathcal{N}((1-t)x_0 + tx_1, \sigma_{\text{path}}^2 I), \quad x_t = (1-t)x_0 + tx_1 + \sigma_{\text{path}} \eta, \quad \eta \sim \mathcal{N}(0, I). \quad (2.24)$$

Differentiating the conditional mean with respect to t gives the corresponding target vector field

$$u_t(x | x_0, x_1) = x_1 - x_0, \quad (2.25)$$

which is independent of x — exactly the straight-line constant-velocity interpolation between the noise sample and the data sample. The training objective becomes

$$\mathcal{L}_{\text{I-CFM}} = \mathbb{E}_{t, x_0, x_1, \eta} \left[\|v_\theta((1-t)x_0 + tx_1 + \sigma_{\text{path}}\eta, t) - (x_1 - x_0)\|^2 \right]. \quad (2.26)$$

The path noise $\sigma_{\text{path}} > 0$ “thickens” the otherwise-singular conditional paths and regularises the vector-field fit, following (author?) [24]; FlowCast uses $\sigma_{\text{path}} = 0.01$. A

derivation of (2.25) from (2.24) and an explicit comparison with the curved probability flow of DDPM are in Appendix B.3, §C.12.



FlowCast. (author?) [16] introduced FlowCast, the first fully probabilistic application of I-CFM to precipitation nowcasting. FlowCast trains a single U-Net backbone — in the original paper an Earthformer-style cuboid-attention network operating in a VAE latent space — to regress the straight-line vector field $v_\theta(x_t, t, c)$ conditioned on a past-observation bundle c . Sampling uses a 10-step Euler integrator of $\dot{x}_t = v_\theta(x_t, t, c)$. In a direct ablation on the SEVIR benchmark, FlowCast at 1 ODE step beat DDIM at 100 steps on CRPS and CSI, and matched or exceeded the state-of-the-art probabilistic nowcasting baselines CasCast and PreDiff at a fraction of the inference cost. Two features make the method relevant to this thesis: (i) the straight-line ODE is by construction the trajectory any one-step sampler is trying to approximate, so the performance gap between 1-step and 10-step sampling is much smaller than for curved diffusion ODEs; and (ii) training is simulation-free and requires no teacher — FlowCast learns its velocity field directly from data in a single training run. In Section 3.5 we adapt FlowCast to the StormCast residual pipeline and compare it directly to the EDM diffusion baseline on the same 2022 Taiwan validation year.

2.9 Flow Matching versus Diffusion: Why Straight-Line Paths Help

Both score-based diffusion and Conditional Flow Matching learn a deterministic ODE that carries a Gaussian prior sample to a data sample; the difference is the shape

of the path that ODE follows, and that shape is what governs how few steps a sampler can use. It is worth stating the contrast explicitly, because it is the single fact that motivates the choice of FlowCast over diffusion in this thesis.



The diffusion path is curved. The EDM PF-ODE of Equation (2.13) has a velocity $-\sigma \nabla_x \log p_\sigma(x)$ that depends on the score of the noised marginal, which itself changes nonlinearly with σ . The trajectory $\sigma \mapsto x(\sigma)$ is therefore curved, and a low-order integrator (Euler, or even Heun) accumulates a truncation error that grows as the step size grows. This is why high-quality diffusion sampling sits in the 20–50-NFE band: below it, the integrator’s curvature error becomes visible, and it shows up first on the sparse, heavy-tailed precipitation channel where the score is most ill-conditioned at small σ .

The I-CFM path is a straight line. Independent CFM (Section 2.8) defines its conditional probability path as the straight-line interpolation $x_t = (1 - t)x_0 + tx_1$ between a prior sample x_0 and a data sample x_1 , with the constant target velocity $u_t = x_1 - x_0$. Along a single such conditional path the trajectory has zero curvature, so an explicit Euler step incurs no truncation error on the conditional path; what error remains comes only from the fact that the learned marginal velocity field averages over many crossing conditional paths. In practice this makes the gap between 1-step and 10-step flow-matching sampling far smaller than the gap between 1-step and 25-step diffusion sampling — the empirical claim that (author?) [16] verified on radar nowcasting and that Chapter 4 re-tests on the Taiwan RWRP domain.

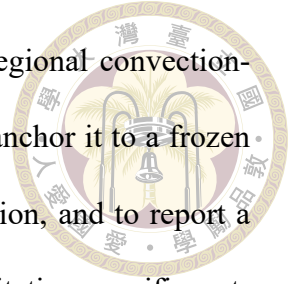
No teacher, one training run. A third practical difference is organisational. The standard way to make diffusion cheap at inference is distillation: train a fast student to imitate

a slow, pretrained teacher's sampling output (progressive distillation [18], consistency distillation [22], adversarial distillation [19]). Distillation needs a converged teacher, a second training stage, and careful matching of the teacher's sampler and preconditioning. Rectified flow [11] showed that one can instead reshape the path itself so that straight-line integration suffices. FlowCast takes this route to its simplest conclusion: it trains the straight-line velocity field directly from data in a single run, with no teacher and no second stage. This thesis therefore compares a single-stage flow-matching forecaster against the diffusion model it would otherwise have to distil, rather than comparing distillation schemes among themselves.

2.10 Prior Generative Models for Precipitation Nowcasting

Generative precipitation modelling has converged on the same conclusion from several directions. DGMR [15] used a conditional GAN and was the first to argue convincingly that adversarial/generative training, not deterministic regression, is what produces skilful sharp radar nowcasts. Diffusion-based nowcasters and downscalers followed: GenCast [14] at global scale and StormCast [12] at regional convection-permitting scale both use score-based diffusion residuals, and NVIDIA's CorrDiff line applies an EDM super-resolution residual to km-scale downscaling. The common cost of all of these is the multi-step diffusion sampler. Flow matching is the most recent entrant: (author?) [16] introduced FlowCast as the first fully-probabilistic Independent-CFM nowcaster and reported that it matched or beat state-of-the-art diffusion nowcasters (CasCast, PreDiff) at a small fraction of their NFE on the SEVIR radar benchmark. To the best of our knowledge, the

present thesis is the first to adapt conditional flow matching to a regional convection-permitting forecaster (as opposed to a radar-to-radar nowcaster), to anchor it to a frozen NWP-style regression mean in the StormCast two-stage decomposition, and to report a head-to-head flow-matching-versus-diffusion comparison with precipitation-specific metrics on the Taiwan domain.



2.11 Evaluation of Precipitation Forecasts

Pixelwise RMSE alone is a poor measure of precipitation-forecast quality because it rewards smooth, low-amplitude predictions that miss extreme events. We adopt a suite of complementary metrics, each introduced briefly here and defined formally in Chapter 4.

Categorical skill scores. For a precipitation threshold τ , every pixel in the forecast and observation is classified as wet ($\geq \tau$) or dry, producing a 2×2 contingency table with hits H , misses M , false alarms F , and correct negatives. The Critical Success Index $CSI = H / (H + M + F)$ is the standard operational score. The frequency bias $(H + F) / (H + M)$ diagnoses over- or under-prediction.

Fractions Skill Score. (author?) [17] introduced FSS to handle the double-penalty problem: a forecast that is slightly displaced in space but correct in structure should not be penalised as heavily as one that is wrong everywhere. FSS computes the fraction of pixels exceeding τ in a neighbourhood of radius n around each point and compares forecast and observation fractions.

Spectral diagnostics. The radially averaged logarithmic power spectral density (log-PSD) of the forecast and observation fields quantifies sharpness. A deterministic model that produces blurry outputs will have a log-PSD that drops off too quickly at high wavenumbers; a generative model that matches the data distribution will track the observation spectrum.

Rollout error growth. For an autoregressive forecaster, single-step error understates the operational risk. We therefore report errors at rollout horizons of 1, 3, 6, and 12 hours, which captures the cumulative effect of single-step residual-head error on autoregressive predictions.

Probabilistic scores (optional). For ensemble forecasts we additionally report the continuous ranked probability score (CRPS) and rank histograms for qpepre.





Chapter 3 Method

This chapter describes what was actually built. Section 3.1 states the residual-forecasting problem in precise terms. Section 3.2 describes the Taiwan RWRP dataset and its normalisation. Section 3.3 documents the underlying data sources and the preprocessing pipeline that turns them into the training Zarr store. Section 3.4 recaps the frozen regression mean and the EDM diffusion residual that serves as our reference baseline. Section 3.5 describes FlowCast, the Conditional Flow Matching residual that is the method of this thesis. Section 3.6 describes the dataset-specific loss modifications for the precipitation channel, which are shared by the diffusion baseline and FlowCast. Section 3.7 documents the training infrastructure.

3.1 Problem Setup

We adopt the StormCast two-stage decomposition throughout. A frozen deterministic regression network F_ξ produces a conditional mean

$$M_t = F_\xi(X_{t-1}, S_t, I), \quad (3.1)$$

and a generative residual head models the residual $R_t = X_t - M_t$ conditioned on a bundle assembled from the previous state, the regression mean, and the invariants. The regression

mean is the same frozen checkpoint for every method in this thesis, so the only object that varies is the generative residual head.



We compare two interchangeable residual heads, holding the dataset, the regression mean M_t , the backbone architecture, and the data scale σ_{data} fixed:

- **EDM diffusion (baseline).** A frozen-recipe EDMPrecond SongUNet D_ψ models the residual as a score-based diffusion and samples it by integrating the PF-ODE from σ_{max} to σ_{min} with a Heun sampler ($N \approx 13$ steps, ≈ 25 NFE per hourly step). This is the reference against which FlowCast is measured; it is the residual head of the original StormCast.
- **FlowCast (this thesis).** A FlowCastPrecond SongUNet velocity field v_θ models the residual as a straight-line Conditional Flow Matching ODE and samples it with K explicit Euler steps (K NFE per hourly step), where we sweep $K \in \{1, 2, 4, 8, 10, 15, 20\}$.

In both cases the full prediction at each step is $\hat{X}_t = M_t + \hat{R}_t$, and the model is rolled out autoregressively, with the previous generated state feeding the next step's regression input. The central empirical question of the thesis is whether the single-run, teacher-free FlowCast head matches or beats the diffusion head at a substantially smaller NFE (and wall-clock) budget, at a matched training-sample budget so that the comparison reflects the generative trajectory rather than training compute. FlowCast does not distil D_ψ and never queries it during training; the two heads are trained independently on the same data.

3.2 Dataset: Taiwan RWRF



The training data live in a Zarr store under the project directory (see Appendix A.4).

The store has three top-level groups.

LowRes conditioning S_t . Shape $(T, 24, 224, 128)$, where T is the hourly time index and the 24 channels are mean sea-level pressure, two-metre temperature, ten-metre winds, and the specific humidity, temperature, and zonal and meridional winds at four pressure levels (1000 hPa, 850 hPa, 500 hPa and 250 hPa). These channels are low-resolution re-gridded ERA5 fields that serve as synoptic-scale boundary conditioning.

HighRes target X_t . Shape $(T, 4, 224, 128)$, with four channels: two-metre temperature (t2m), ten-metre zonal and meridional winds (u10, v10), and hourly accumulated precipitation (qpepre). The horizontal grid covers Taiwan from 21.6° to 25.6°N and 119.75° to 122.25°E at approximately 2 km resolution. The dataset splits into 21,216 training hours (2019-08 through 2021-12) and 8,760 validation hours (calendar year 2022).

Invariants I . Shape $(2, 224, 128)$, channels land-sea mask and orography. These are broadcast across time and concatenated into the conditioning bundle.

Normalisation. Per-channel means and standard deviations are computed on the training split and stored in the Zarr metadata. The HighRes statistics are

$$\mu_{\text{HR}} = (295.98, -1.58, -2.65, 0.182), \quad \sigma_{\text{HR}} = (5.61, 3.84, 5.66, 9.12), \quad (3.2)$$

which show that qpepre has a near-zero mean with standard deviation an order of magnitude larger than its mean — a direct fingerprint of its heavy-tailed, sparse distribution. Training and validation losses are computed on normalised fields; metrics in Chapter 4 are reported on denormalised fields with physical units.

Cleaned variant used for the reported experiments. The shapes above describe the original store. All FlowCast-versus-diffusion runs in this thesis are trained and evaluated on a cleaned re-export of the same data, in which the precipitation channel is encoded as $\log(1 + x)$ before standardisation (the `qpepre_log1p` option of the loader) and the spatial domain is centre-cropped to a U-Net-friendly 192×96 grid. The cleaned store covers 18,765 training hours (2019-08-01 through 2021-12-31) and 8,759 validation hours (calendar year 2022). The $\log(1 + x)$ encoding is the single most important preprocessing choice for the precipitation channel: it compresses qpepre’s dynamic range from a raw standard deviation of 9.12 mm h^{-1} to roughly 0.37 in log space, which makes the standardised residual far better-conditioned for both residual heads (Section 4.7). Because the spatial crop changes the field of view, RMSE/CRPS magnitudes are not directly comparable to the legacy 224×128 baseline of Section 4.9; we flag this wherever the two are tabulated together. The regression mean and both residual heads are trained on this same cleaned, log1p-encoded store, which is what the “train on the same qpepre encoding” gotcha in Section 4.12 refers to.

3.3 Data Sources and Preprocessing Pipeline

Section 3.2 described the Zarr store consumed by the training loop. This section describes where the underlying fields come from and how the preprocessing pipeline in

data_preprocessing/ assembles them. The design follows a low-resolution guidance, high-resolution learning strategy: a coarse global reanalysis provides synoptic-scale conditioning, and a convection-permitting regional model plus a radar-derived precipitation product provides the fine-scale target.



3.3.1 Raw data sources

ERA5 (low-resolution input, S_t). The ECMWF ERA5 reanalysis [5] at approximately 25 km spatial resolution supplies the synoptic-scale environmental state. From ERA5 we take four surface variables (mean sea-level pressure `mslp`, two-metre temperature `t2m`, and ten-metre winds `u10`, `v10`) and five upper-air variables (specific humidity `q`, temperature `t`, zonal wind `u`, meridional wind `v`, and geopotential height `z`) at four pressure levels {1000, 850, 500, 250} hPa, for a total of $4 + 5 \times 4 = 24$ channels. These are the 24 LowRes channels listed in Section 3.2.

RWRF (high-resolution target, part of X_t). The Radar-assimilated Weather Research and Forecasting (RWRF) numerical model provides convection-permitting (≈ 2 km) analyses over Taiwan. We use hourly RWRF output (`wrfout_d01*_interp`) for the three near-surface dynamical channels: `t2m`, `u10`, `v10`. RWRF contains the fine-scale dynamic and thermodynamic structures that the complex Taiwan terrain imprints on near-surface weather, which is exactly what a regional convective-scale forecaster must learn.

QPEPRE (high-resolution target, precipitation channel). Quantitative Precipitation Estimation (QPEPRE) is a radar-based hourly-rainfall product distributed as fixed-grid text files (`qpepre_YYYYMMDDHHMM-YYYYMMDDHHMM_1_h.txt`), one per hour. Unlike the

Table 3.1: Raw data sources used for training. Low-resolution ERA5 provides synoptic conditioning; RWRF and QPEPRE together supply the convection-permitting target.

	ERA5	RWRF	QPEPRE
Type	Reanalysis	Numerical model	Radar observation
Resolution	≈ 25 km	≈ 2 km	≈ 2 km
Role	LowRes input S_t	HighRes target X_t	HighRes target X_t
Channels used	24 (4 sfc + 5×4 levels)	3 (t2m, u10, v10)	1 (qpepre)
Native format	NetCDF	NetCDF (wrfout)	ASCII text

RWRF fields, which are model outputs, QPEPRE is an observational product and therefore serves as a more faithful ground truth for surface rainfall than any model-forecast precipitation field. It occupies the fourth HighRes channel, qpepre.

Invariants I . Two static fields are extracted once from an RWRF file and broadcast across time: the land–sea mask `lsm` and orography `orog`. These let the network condition on terrain without re-reading static fields at every time step.

Table 3.1 summarises the three sources.

3.3.2 Preprocessing pipeline

The project ships two preprocessing stages under `data_preprocessing/`:

1. `nc_month_to_day/` — split monthly ERA5 NetCDF files into single-hour files.
2. `nc_to_zarr/` — harmonise ERA5, RWRF, and QPEPRE onto a common Taiwan grid and write a StormCast-style Zarr store.

Stage 1: monthly-to-hourly split. ERA5 is distributed as monthly NetCDF files, each containing an hourly time axis. The script `split_era5_monthly_to_daily.py` walks a root directory, opens every file matching a configured pattern, decodes the time axis with

netCDF4.num2date, and writes one file per hour with a canonical {variable}_YYYYMMDDTHH.nc filename. The output uses uniform hours since 1970-01-01 00:00:00 time units so that downstream indexing does not have to handle per-file calendar differences. The split runs in parallel via a multiprocessing.Pool whose worker count is configurable (typically 20 processes on a workstation). Driver: nc_month_to_day/split_era5_monthly_to_daily.py; configuration: nc_month_to_day/config.yaml.

Stage 2: unified NetCDF-to-Zarr assembly. The main pipeline is nc_to_zarr/nc_to_zarr_pipeline driven by config.yaml. It is parameterised by four paths and a domain description:

era5-path, rwrf-path, qpepre-path, output-path

domain-size: [224, 128]

lon-bounds: [119.75, 122.25]

lat-bounds: [21.6, 25.6]

train-ranges / valid-ranges

For each requested split (train, valid, or combined), the pipeline performs the following steps.

1. **Time-stamp generation.** The configured date ranges are expanded into a dense hourly list between the start and end of each range.
2. **File indexing.** Three in-memory indices are built — one per source — that map timestamps to absolute file paths. For ERA5 this is a nested {variable → {date-key → path}} dictionary built by a regex on filenames; for RWRF it is {folder-name → path}; for QPEPRE it is {time-key → path} derived from the qpepre_{start}-{end}_1_h.txt filename convention. When multiprocessing is enabled the indices

are serialised to a single JSON file `.shared_index.json` in the output directory so that worker processes can mmap-read the index instead of receiving a pickled copy.



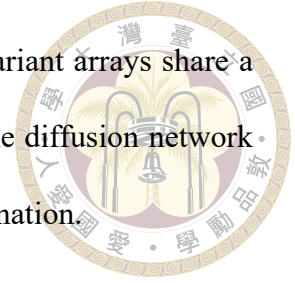
3. **Combined validity precomputation.** Before processing anything, the pipeline walks the timestamp list once and flags every hour for which any of the required ERA5 variables, RWRf file, or QPEPRE file is missing. Only timesteps that pass this joint check are processed, and the Zarr store carries an explicit per-timestamp valid mask so that downstream code can skip gaps without re-scanning the raw files.

4. **Per-timestep field extraction.**

- ERA5: open the NetCDF, select the target time index, select the correct pressure level from the level dimension (for upper-air variables), crop to the latitude/longitude bounds, and resample to the configured 224×128 target grid.
- RWRf: read XLAT and XLONG to cache the native latitude/longitude, find the target Time index from the Times character variable, slice each requested variable to the domain, optionally apply per-variable transforms, and resample to the target grid.
- QPEPRE: load the text file, parse it into a {lat, lon, rain} scatter, grid it onto the native QPEPRE resolution, and resample to the 224×128 target grid. Because QPEPRE is already an observational product, we use it directly as the qpepre HighRes channel rather than the RWRf precipitation field.

5. **Grid harmonisation.** All three sources are resampled to a shared $H \times W = 224 \times 128$ grid covering lat $[21.6, 25.6]^\circ\text{N}$, lon $[119.75, 122.25]^\circ\text{E}$ at approximately 2 km

spacing. This guarantees that the LowRes, HighRes, and invariant arrays share a pixel-aligned coordinate system, which is essential because the diffusion network conditions on LowRes and invariants by channel-wise concatenation.



6. **Streaming statistics.** As each training sample is filled into the data cube, a `StreamingStats` accumulator updates per-channel running means and standard deviations in single-pass Welford form. After the train split finishes, the stats are written to `stats/means.npy` and `stats/stds.npy` alongside the Zarr store. These are the normalisation statistics quoted in Equation 3.2.
7. **Zarr writing.** The assembled cube is written as an `xarray.Dataset` with dimensions `(time, channel, y, x)`, chunked along time and space for efficient random access, and consolidated (`zarr.consolidate_metadata`) so that the training loader can open the store with a single metadata read. Invariants are written once, without a time dimension, as a separate Zarr store under `invariants/invariants.zarr`.

Output layout. A successful run produces the tree

```
<output-path>/
```

```
LowRes/
```

```
stormcast_test_train.zarr      # (T_train, 24, 224, 128)
```

```
stormcast_test_valid.zarr     # (T_valid, 24, 224, 128)
```

```
stats/{means.npy, stds.npy, channels.txt}
```

```
HighRes/
```

```
stormcast_test_train.zarr     # (T_train, 4, 224, 128)
```

```
stormcast_test_valid.zarr     # (T_valid, 4, 224, 128)
```

```
stats/{means.npy, stds.npy, channels.txt}
invariants/
invariants.zarr # (2, 224, 128)
```

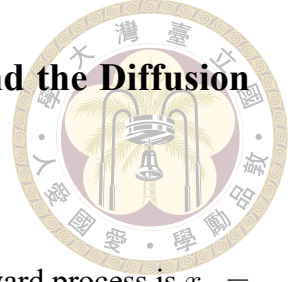


which matches the layout documented in `zarr_metadata.json` and consumed by `data_loader_rwrf_era`

For the experiment used in this thesis the pipeline produces $T_{\text{train}} = 21,216$ hourly samples (2019-08-01 through 2021-12-31) and $T_{\text{valid}} = 8,760$ hourly samples (calendar year 2022).

Why this matters here. Three features of this pipeline are load-bearing for the work in this thesis:

- Pixel-aligned multi-source grids. The diffusion baseline and FlowCast both consume conditioning and target on the same grid, so residual losses can be computed pixel-wise without any on-the-fly re-gridding.
- Explicit valid mask. Roughly 10–15 % of hours fail the joint-validity check (usually a missing QPEPRE file). Both loaders and metrics respect the mask, so downstream scores are never contaminated by NaN-filled cells.
- Frozen training-split statistics. Because the Zarr store commits to a single pair of $(\mu_{\text{HR}}, \sigma_{\text{HR}})$ at write time, both residual heads see identical normalised inputs — a prerequisite for the σ_{data} -matching discussion in Section 3.4.



3.3.3 Target Distributions: Sparsity, Gaussianity, and the Diffusion Prior

Diffusion models place a Gaussian prior on their target: the forward process is $x_\sigma = x_0 + \sigma \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I)$, and the denoiser is trained under the implicit assumption that the marginals of x_0 are reasonably well-behaved relative to that noise scale. How closely each of the four HighRes channels actually conforms to this assumption directly shapes how the generative residual head must be regularised. Figure 3.1 shows per-channel histograms (with a fitted Gaussian overlay) and Gaussian QQ plots computed over the full training split ($n = 6.08 \times 10^8$ pixels per channel). The streaming statistics are summarised in Table 3.2.

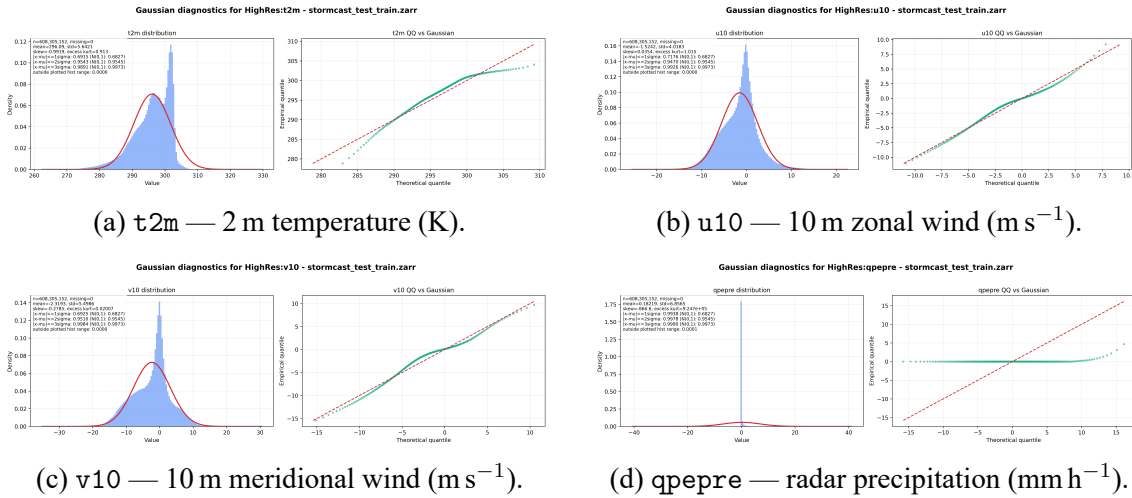


Figure 3.1: Per-channel distribution and Gaussian QQ plots for the four HighRes target channels over the 2.5-year training split. Left subpanels: empirical density (red) with a Gaussian fit of matched (μ, σ) (blue). Right subpanels: empirical quantiles against theoretical Gaussian quantiles; the red dashed line is $y = x$ for a perfect Gaussian. Generated by `data_preprocessing/inspect_tools/plot_high_res_vars.py`.

The three near-surface state channels are approximately Gaussian. u10 is the closest: almost zero skewness, mild leptokurtosis, and a QQ plot that tracks the diagonal out to roughly $\pm 4\sigma$ before fanning into the heavy tails associated with gust events. v10 is almost symmetric with essentially zero excess kurtosis. t2m is the least symmetric of the three:

Table 3.2: Summary statistics of the four HighRes target channels over the training split. A standard Gaussian has skew = 0, excess kurt = 0, and concentrates 68.3/95.5/99.7% of its mass within 1/2/3 σ .

Channel	Mean	Std	Skewness	Excess kurt.	% in 1 σ	% in 3 σ
t2m	296.09	5.64	−0.99	0.91	69.2	98.9
u10	−1.52	4.02	0.04	1.01	71.8	99.3
v10	−2.32	5.50	−0.28	0.02	69.3	99.8
qpepre	0.18	6.86	−866.6	9.25×10^5	99.4	99.9

left-skewed (skew = −0.99) with a secondary mode around 280 K corresponding to winter cold surges over northern Taiwan and the surrounding sea. For all three channels the empirical 1 σ /2 σ /3 σ coverage agrees with the Gaussian reference to within a percentage point, which is exactly the regime for which the EDM noise schedule with $\sigma_{\text{data}} = 0.5$ was designed: the generative residual head sees each channel as a mild deformation of a unit-variance Gaussian after normalisation, and the standard ℓ_2 objective is a reasonable fit.

qpepre is a different object entirely. Its marginal is dominated by an atom at zero — the overwhelming majority of pixel-hours are dry — with a long, very sparse tail of heavy rainfall events. The raw statistics make this quantitative: the coverage inside $\pm 1\sigma$ is 99.4% rather than the Gaussian 68.3%, which is the signature of a near-delta distribution whose standard deviation is inflated entirely by the tail. The empirical moments reported in Table 3.2 (skewness ≈ -867 , excess kurtosis $\approx 10^6$) are formally enormous because the raw qpepre field uses a −9999 sentinel for missing observations that is only filtered at the next pipeline stage; the physical tail above zero is still extreme (excursions beyond 100 mm h^{−1} occur during typhoon landfalls), but the numbers headline the practical message: the qpepre histogram looks nothing like a Gaussian, and the QQ plot sits on a flat shelf near zero that peels almost vertically at the upper end.

This mismatch has three consequences for residual generative modelling that recur

throughout the rest of this thesis:

1. Score matching on raw qppepre is ill-posed at small σ . When the target distribution is almost a point mass at zero, the diffusion score $\nabla_x \log p_\sigma(x)$ diverges as $\sigma \rightarrow \sigma_{\min}$ — one reason the EDM baseline needs many small- σ steps to render precipitation. Learning the residual relative to the regression mean M_t partially absorbs this, but the residual inherits the same atom-plus-tail shape, and the $\log(1+x)$ encoding of Section 3.6 is what finally compresses its dynamic range. The straight-line FlowCast velocity field has no score term and is correspondingly better-behaved on this channel.
2. Pixel-wise losses undersell the tail. Because dry pixels vastly outnumber rainy ones, an ℓ_2 loss converges to a “slightly wet everywhere” solution that scores well on RMSE but destroys the convective structure we actually care about. This is the motivation for the radial log-PSD spectral regulariser applied specifically to qppepre in the FlowCast loss (Section 3.6), and for the precipitation-specific metrics (CSI, FSS, HSS, frequency-of-exceedance at fixed rainfall thresholds) reported in Chapter 4.
3. The channel budget matters. Because only one of the four output channels is pathological, per-channel loss weights β_k are a natural knob: the three near-Gaussian fields can be trained with a standard denoising objective while qppepre receives heavier regularisation and, if needed, an asinh or $\log(1+x)$ transform to compress its dynamic range before the noise is added. Having only four target channels (versus the 99 of global StormCast) makes such per-channel bespoke treatment computationally trivial.

In short, Figure 3.1 is the quantitative justification for every “qppepre is the hard chan-

nel” design decision that follows: three channels live comfortably inside the Gaussian prior that diffusion assumes, and one does not.



3.4 Regression Mean and the EDM Diffusion Baseline

The reference model is the two-stage StormCast described in Section 2.3, with the EDM hyperparameters of Section 2.6. It supplies both the frozen regression mean that FlowCast reuses unchanged and the diffusion residual that FlowCast is compared against.

- **Regression** F_ξ . A StormCastUNet taking (X_{t-1}, S_t, I) as input and predicting M_t .

The checkpoint is used without modification or further training, and is shared by the diffusion baseline and FlowCast so that both residual heads model a residual relative to the same mean.

- **Diffusion baseline** D_ψ . An EDMPrecond SongUNet trained to denoise the residual

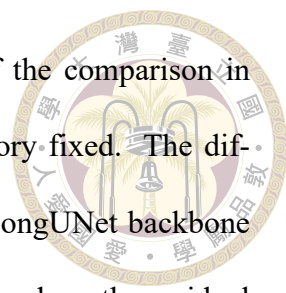
$R_t = X_t - M_t$ conditioned on $c_t = \text{concat}(X_{t-1}, S_t, M_t, I)$. EDM hyperparameters:

$\sigma_{\min} = 0.002$, $\sigma_{\max} = 80$, $\sigma_{\text{data}} = 0.5$, $\rho = 7$; sampling is the Heun integrator of

Section 2.6. The baseline is trained from scratch on the same cleaned dataset as

FlowCast and evaluated at the same ≈ 2 M-training-sample budget (the nearest saved checkpoint; see Section 4.1).

Both networks are instantiated through the factory `get_preconditioned_architecture` in `stormcast/utils/nn.py`, which selects the EDM or FlowCast preconditioning wrapper by name. The conditioning bundle is assembled by `build_network_condition_and_target`, which handles the channel-wise concatenation and the one-hot time-offset encoding used in the rollout.



Why the baseline matters for a fair comparison. The value of the comparison in Chapter 4 rests on holding everything except the generative trajectory fixed. The diffusion baseline and FlowCast share the dataset, the frozen F_ξ , the SongUNet backbone (`channel_mult = [1, 2, 2, 2, 2]`), and the data scale $\sigma_{\text{data}} = 0.5$ that anchors the residual statistics. The EDM data scale matters here for a subtle reason: FlowCast standardises its residual by exactly the same σ_{data} (Section 3.5), so the per-channel loss scaling and the regression-mean anchoring are numerically identical across the two heads. A mismatched σ_{data} would silently shift the residual distribution one head sees relative to the other and make any quality difference uninterpretable.

3.5 The FlowCast Residual (Conditional Flow Matching)

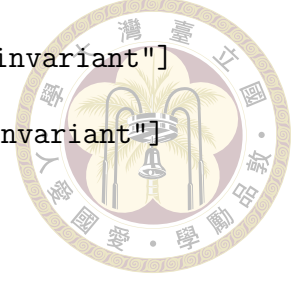
FlowCast replaces the diffusion baseline’s curved PF-ODE with a simulation-free straight-line flow: a single network is trained from scratch to regress the target vector field $u_t = x_1 - x_0$ of Independent CFM (Section 2.8), and sampling is a handful of Euler steps. FlowCast is not a distillation method — it never queries the EDM diffusion model and uses no teacher at all — it is a one-training-run residual head that learns its velocity field directly from data. The same StormCast residual decomposition is preserved: the deterministic regression mean M_t is produced by the frozen F_ξ , and FlowCast learns only the residual $R_t = X_t - M_t$ conditioned on a bundle c_t assembled by `build_network_condition_and_target` in `stormcast/utils/nn.py`.

Conditioning bundle. The conditioning channels FlowCast sees differ slightly from the EDM baseline’s. The training config `stormcast/config/model/flowcast.yaml` sets

```

regression_conditions: ["state", "background", "invariant"]
diffusion_conditions: ["state", "regression", "invariant"]

```



so the frozen regression network F_ξ is fed the full (X_{t-1}, S_t, I) as before, but the flow-matching network itself is conditioned only on

$$c_t = \text{concat}(X_{t-1}, M_t, I), \quad (3.3)$$

with the low-resolution background S_t deliberately omitted from the direct conditioning path. The synoptic-scale signal in S_t therefore enters FlowCast only through the regression mean $M_t = F_\xi(X_{t-1}, S_t, I)$. This trims the conditioning-input channel count by 24 at zero cost to the flow matching objective, because M_t already summarises what the residual needs to know about S_t ; keeping "regression" in the diffusion-conditions list is what keeps the FlowCast residual anchored to exactly the same M_t the EDM diffusion baseline is anchored to, which is what makes the head-to-head comparison of Chapter 4 fair.

Standardised-residual pipeline. FlowCast operates on the residual in a standardised space. The loss (`stormcast/utils/flowcast_loss.py`) divides the raw residual by $\sigma_{\text{data}} = 0.5$ before building the flow trajectory, and the sampler (`stormcast/utils/nn.py:flowcast_model_forward`) multiplies the integrated state by σ_{data} on output. Keeping σ_{data} matched to the EDM baseline means the regression-mean anchoring statistics and the per-channel loss scaling are identical across both residual heads. Concretely, with $x_1 = R_t/\sigma_{\text{data}}$ and $x_0 \sim \mathcal{N}(0, I)$, the I-CFM interpolated state is

$$x_t = (1 - t) x_0 + t x_1 + \sigma_{\text{path}} \eta, \quad t \sim \mathcal{U}(t_\varepsilon, 1 - t_\varepsilon), \quad \eta \sim \mathcal{N}(0, I), \quad (3.4)$$

with $\sigma_{\text{path}} = 0.01$ from (author?) [24] and $t_\varepsilon = 10^{-5}$ clamping the endpoints. The small $\sigma_{\text{path}}\eta$ perturbation thickens the otherwise-Dirac conditional path $p_t(x_t \cdot | x_0, x_1)$ and is what separates I-CFM from the original deterministic Optimal-Transport CFM of (author?) [10].

Velocity predictor. The FlowCast network is the FlowCastPrecond module at `stormcast/` `utils/flowcast_precond.py`, instantiated through the factory call `get_preconditioned_architecture(...)` at `stormcast/` `utils/nn.py`. The factory wraps a SongUNet with `channel_mult=[1, 2, 2, 2, 2]`, identical to the EDM baseline’s backbone. Unlike EDMPrecond there are no $c_{\text{skip}}/c_{\text{in}}/c_{\text{out}}$ scalings — the network output is the velocity estimate:

$$v_\theta(x_t, t, c_t) = \text{SongUNet}(\text{concat}[x_t, c_t], t \cdot \tau), \quad (3.5)$$

where $\tau = 1000$ (the `time_scale` field in `stormcast/config/model/flowcast.yaml`) rescales the flow time $t \in [0, 1]$ before it enters the SongUNet’s positional-embedding layer. The SongUNet positional embedding was designed for diffusion-timestep-valued inputs with dynamic range of order 10^2 – 10^3 ; feeding raw $t \in [0, 1]$ would collapse the embedding to a nearly-constant signal. $\tau = 1000$ restores enough dynamic range for the embedding to carry useful time information without changing the SongUNet itself, so no architectural changes from the diffusion baseline’s backbone are needed.

Loss. The training loss is the standardised-space I-CFM regression

$$\mathcal{L}_{\text{FC}} = \mathbb{E}[\|v_\theta(x_t, t, c_t) - (x_1 - x_0)\|_{2,\beta}^2] + \lambda_{\text{spec}} \mathcal{L}_{\text{spec}}(\hat{x}_1, x_1), \quad (3.6)$$

Algorithm 1 FlowCast training step (I-CFM on the standardised residual)

Require: frozen F_ξ , FlowCast velocity field v_θ , schedule $(\sigma_{\text{data}}, \sigma_{\text{path}}, t_\varepsilon)$, channel weights β , spectral weight λ_{spec}

- 1: sample mini-batch $\{(X_{t-1}, S_t, X_t)\}$; form $M_t \leftarrow F_\xi(X_{t-1}, S_t, I)$, $R_t \leftarrow X_t - M_t$
- 2: build conditioning $c_t \leftarrow \text{concat}(X_{t-1}, M_t, I)$ $\triangleright$ note: S_t reaches v_θ only via M_t
- 3: standardise: $x_1 \leftarrow R_t / \sigma_{\text{data}}$
- 4: sample $x_0 \sim \mathcal{N}(0, I)$, $t \sim \mathcal{U}(t_\varepsilon, 1 - t_\varepsilon)$, $\eta \sim \mathcal{N}(0, I)$
- 5: build interpolant: $x_t \leftarrow (1 - t) x_0 + t x_1 + \sigma_{\text{path}} \eta$
- 6: target field: $u_t \leftarrow x_1 - x_0$
- 7: predict: $\hat{v} \leftarrow v_\theta(x_t, t, c_t)$
- 8: pointwise: $\mathcal{L}_{\text{pw}} \leftarrow \|\hat{v} - u_t\|_{2, \beta}^2$
- 9: implied clean: $\hat{x}_1 \leftarrow x_t + (1 - t) \hat{v}$
- 10: spectral: $\mathcal{L}_{\text{spec}} \leftarrow \frac{1}{|\mathcal{K}|} \sum_{\kappa} |\log P_{\hat{x}_1^{\text{qpepre}}}(\kappa) - \log P_{x_1^{\text{qpepre}}}(\kappa)|$
- 11: $\mathcal{L} \leftarrow \mathcal{L}_{\text{pw}} + \lambda_{\text{spec}} \mathcal{L}_{\text{spec}}$
- 12: AdamW step on θ ; EMA update $\theta^- \leftarrow 0.999 \theta^- + 0.001 \theta$

where $\|\cdot\|_{2, \beta}$ is the per-channel-weighted L2 norm from Section 3.6 with $\beta = (1, 1, 1, 2)$ (the `channel_weights` field), $\hat{x}_1 = x_t + (1 - t) v_\theta$ is the flow’s implied clean-residual estimate at the current time, and $\mathcal{L}_{\text{spec}}$ is the radial log-PSD regulariser of Section 3.6 with `spectral_channels` = {qpepre} and $\lambda_{\text{spec}} = 0.1$. Operating the spectral term on the implied-clean field $\hat{x}_1$ rather than on the raw velocity keeps the spectrum interpretable. Algorithm 1 summarises the per-step update.

Optimisation. Following the FlowCast paper we use the AdamW optimiser with $\beta_{\text{Adam}} = (0.9, 0.999)$, learning rate 5×10^{-4} , and weight decay 10^{-4} on all parameters. The learning-rate schedule is a 4,000-step linear warmup ($\sim 1\%$ of the training budget) into a cosine decay to $0.01 \times 5 \times 10^{-4} = 5 \times 10^{-6}$ over the remaining steps, implemented inline in `trainer_flowcast.py`. Gradients are globally clipped at 1.0, and per-parameter NaN gradients are sanitised with `torch.nan_to_num` (a safety net borrowed from the EDM trainer). Training runs for a total of 400,000 optimiser steps at a global batch size of 64, split across the available GPUs through PyTorch’s `DistributedDataParallel`; the per-GPU micro-batch is `batch_size_per_gpu = "auto"`, which the trainer resolves to

64/ W for W workers.



EMA shadow network. A second `FlowCastPrecond` instance is kept as an exponential-moving-average shadow of the online weights, with a fixed decay of 0.999. The shadow is updated after every optimiser step by `ExponentialMovingAverage.update` and copied into the shadow network by `apply_shadow(stormcast/utils/ema.py)`. A single fixed decay suffices throughout training because the FlowCast target — the straight-line velocity field — is stationary; there is no shrinking time discretisation for the decay to track. The shadow weights, not the online weights, are used at validation and inference time, and are saved to `ema_state.pt` alongside each regular checkpoint. Loading the raw `checkpoint_*.pt` instead of `ema_state.pt` yields visibly noisier samples.

Sampling. Generation integrates the learned ODE $\dot{x}_t = v_\theta(x_t, t, c_t)$ from $t = 0$ to $t = 1$ with a fixed-step solver. The implementation `flowcast_model_forward` in `stormcast/`
`utils/nn.py` exposes two choices. The default is explicit Euler with K steps:

$$x_{t+\Delta t} = x_t + \Delta t v_\theta(x_t, t, c_t), \quad \Delta t = 1/K, \quad x_{t=0} \sim \mathcal{N}(0, I), \quad (3.7)$$

which costs K NFEs. The alternative `solver="midpoint"` is a single-stage Runge–Kutta second-order scheme:

$$v_{1/2} = v_\theta(x_t, t, c_t), \quad \tilde{x} = x_t + \frac{1}{2}\Delta t v_{1/2}, \quad x_{t+\Delta t} = x_t + \Delta t v_\theta(\tilde{x}, t + \frac{1}{2}\Delta t, c_t), \quad (3.8)$$

which costs $2K$ NFEs but has $O(\Delta t^2)$ local truncation error. The sampler finishes by de-standardising: $\hat{R}_t = \sigma_{\text{data}} x_{t=1}$ and $\hat{X}_t = M_t + \hat{R}_t$. Validation inside the training loop uses $K = \text{valid_num_steps} = 10$ Euler steps, while the main experiments sweep

$K \in \{1, 2, 4, 8, 10, 15, 20\}$ and place each cost point against the ≈ 25 -NFE EDM Heun baseline on the NFE/wall-clock-quality trade-off of Chapter 4.



Why a straight-line flow is the right residual head. Three properties of I-CFM make FlowCast attractive against the diffusion baseline rather than merely a faster imitation of it. First, the straight-line probability path is by construction what a one-step deterministic generator is trying to approximate, so the FlowCast $K = 1$ and $K = 2$ numbers are a strong reference point for the best a few-step ODE sampler trained on the same data can do — and the gap to the $K = 10$ sampler is small (Chapter 4), unlike the steep degradation a diffusion sampler suffers below ≈ 20 NFE. Second, the training is teacher-free and single-stage, so FlowCast avoids the second training run, the sampler/preconditioning matching, and the EMA-coupling subtleties that a distillation of the diffusion baseline would require. Third, and specific to precipitation, (author?) [16] report that I-CFM is noticeably more stable than score matching on sparse heavy-tailed fields — a claim directly relevant to the qpepre channel and one we re-test on the Taiwan domain.

Implementation pointers. Entry point `stormcast/train_flowcast.py`; training loop `stormcast/utils/trainer_flowcast.py` (`flowcast_training_loop`); loss `stormcast/`
`utils/flowcast_loss.py` (`FlowCastLoss`); preconditioner `stormcast/utils/flowcast_precond.py`
 (`FlowCastPrecond`); EMA helper `stormcast/utils/ema.py` (`ExponentialMovingAverage`);
 configs `stormcast/config/flowcast.yaml`, `stormcast/config/model/flowcast.yaml`,
`stormcast/config/training/flowcast.yaml`; sampler `flowcast_model_forward`
 in `stormcast/utils/nn.py`. The numerical hyperparameters are tabulated in Appendix 5.4.

3.6 Loss Design for the Precipitation Channel



The four-channel RWRf target makes per-channel loss engineering cheap, and we use this budget to attack the hardest channel, `qppepre`, from two directions. Both terms below are shared between the EDM baseline and FlowCast where applicable; FlowCast uses the channel-weighted L2 form on its velocity regression plus the spectral term on the implied clean residual.

Channel-weighted L2. Let $\beta = (\beta_{t2m}, \beta_{u10}, \beta_{v10}, \beta_{qppepre})$ be a vector of non-negative per-channel weights. The weighted L2 norm used by the FlowCast velocity loss is

$$\|a\|_{2,\beta}^2 = \sum_{k=1}^4 \beta_k \|a_k\|^2, \quad (3.9)$$

where the sum runs over the four output channels and the norm inside each term is the ordinary spatial L2. The default configuration uses $\beta = (1, 1, 1, w_{pr})$, with the precipitation weight w_{pr} exposed as the last entry of `channel_weights` in `stormcast/config/model/flowcast.yaml`. The headline FlowCast run uses $w_{pr} = 2$; Section 4.8 sweeps $w_{pr} \in [1.0, 2.4]$ to locate the precipitation/wind trade-off. A robust Pseudo-Huber variant $\rho_{c_h,\beta}(a, b) = \sum_k \beta_k (\sqrt{\|a_k - b_k\|^2 + c_h^2} - c_h)$ with a small c_h is also available for bounded gradients on extreme tail events, but is not used in the headline run.

Radial log-PSD regulariser. Because RMSE-like losses reward smooth, low-amplitude precipitation outputs, we add a spectral-matching regulariser that penalises the L1 distance

between the radial log-PSD of the predicted and target qpepre fields:

$$\mathcal{L}_{\text{spec}} = \frac{1}{|\mathcal{K}|} \sum_{\kappa \in \mathcal{K}} |\log P_{\text{pred}}(\kappa) - \log P_{\text{target}}(\kappa)|, \quad (3.10)$$



where $P(\kappa)$ is the radially averaged 2D power spectral density at wavenumber κ and $\mathcal{K}$ is the set of non-zero radial wavenumbers on the spatial grid. The regulariser is active only on channels listed in `spectral_channels` and weighted by `spectral_weight` = λ_{spec} (default 0.1). By default we apply it to qpepre only, on the flow's implied clean residual $\hat{x}_1$ rather than the raw velocity (Algorithm 1).

Monitoring for mode collapse. Even with the above terms, sparse heavy-tailed precipitation is an invitation to mode collapse: the model can minimise the weighted L2 by producing a slightly-wet constant field. During training we therefore log the wet-pixel fraction at thresholds $\{0.1, 1.0, 5.0\} \text{ mm h}^{-1}$ and the 95th and 99th percentiles of the output distribution, and compare them to the same quantities on the training batch. A large divergence is a collapse warning.

3.7 Training Infrastructure

All runs use PyTorch with `torchrun --standalone --nnodes=1 --nproc_per_node=N` for multi-GPU training. Configuration is managed with Hydra; experiment overrides are passed on the command line as key=value pairs. Checkpoints are written every M iterations (default $M = 1000$) under

```
StormCast_{method}/{experiment}/0/checkpoints{method}/
checkpoint_*.pt
```

together with CSV loss logs and, every V iterations, validation heatmap and power-spectrum images for the four output channels. Auto-resume restores both the online checkpoint and, for FlowCast, the EMA shadow (`ema_state.pt`) by a `checkpoint_latest.pt` convention in `trainer_flowcast.py`; the EDM baseline restores from `checkpoints_diffusion/`. A run can therefore be restarted with the same command line and will pick up from the last written checkpoint.







Chapter 4 Experiments and Results

This chapter presents the empirical comparison between the FlowCast residual of Chapter 3 and the EDM diffusion baseline. Section 4.1 defines the experimental protocol and the matched-sample-budget convention. Section 4.2 defines the evaluation metrics. Section 4.3 reports the headline FlowCast-versus-diffusion comparison on the 2022 validation year. Section 4.4 reports the NFE / wall-clock-versus-quality trade-off. Section 4.5 drills into precipitation skill by threshold; Section 4.6 reports autoregressive rollout error growth. Sections 4.7 and 4.8 report the $\log_{10}$ -encoding and precipitation-weight ablations. Section 4.9 places both heads against a legacy old-StormCast baseline. Section 4.10 presents qualitative case studies, Section 4.11 catalogues failure modes, and Section 4.12 concludes with a discussion.

4.1 Experimental Protocol

Validation split. All reported metrics are computed on the cleaned 2022 full-year RWRF validation store (8,759 hourly samples; Section 3.2). This contrasts with the stale `valid_dates` shipped in `stormcast/config/inference/stormcast.yaml`, which we override explicitly in every run.

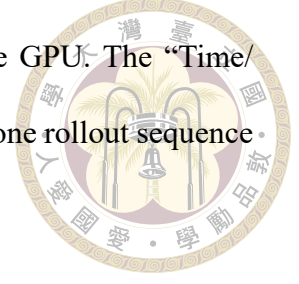
Matched training-sample budget. The fair-comparison unit is training samples seen, not optimiser steps, because the two residual heads use different global batch sizes. The EDM baseline trains at batch 64 and FlowCast at batch 96, so ≈ 2 M samples corresponds to step 31,250 for EDM and step 20,833 for FlowCast. We evaluate each head at the nearest saved checkpoint to this ≈ 2 M-sample anchor (EDM EDMPrecond.0.31000, FlowCast FlowCastPrecond.0.20000 with its EMA shadow). Holding training samples fixed means any quality difference reflects the generative trajectory and not a training-compute advantage.

Single-snapshot noise. Each validation point is a single snapshot. Adjacent validation logs drift by 10–30 %, so differences below ≈ 5 % on a single metric should not be over-interpreted; we call out only gaps that are large relative to this noise floor.

NFE budgets and samplers. The EDM baseline samples with its native Heun schedule (≈ 25 NFE). FlowCast samples with K explicit Euler steps, K NFE; we report $K \in \{10, 15, 20\}$ in the rollout scoreboard and sweep down to $K = 1$ in the inference-step study of Section 4.4. FlowCast always uses its EMA shadow weights, never the online checkpoint.

Rollouts and ensembling. The scoreboard metrics are computed from autoregressive rollouts driven by `compare_diffusion_vs_flowcast.py`, which denormalises every output (undoing standardisation and the $\log_{1p}$ encoding) so that `qpepre` is scored in mm h^{-1} regardless of the on-disk encoding. CRPS uses a small per-hour ensemble of independent noise seeds; deterministic scores use the ensemble mean.

Hardware. Training: multi-GPU single node. Evaluation: single GPU. The “Time/Seq.” column in the scoreboards is the wall-clock cost of generating one rollout sequence on the evaluation GPU and is the operationally relevant cost figure.



4.2 Evaluation Metrics

All metrics are computed on denormalised fields with physical units (K , m s^{-1} , mm h^{-1}).

RMSE and MAE. For channel k ,

$$\text{RMSE}_k = \sqrt{\frac{1}{|\mathcal{T}| HW} \sum_{t \in \mathcal{T}} \sum_{i,j} (\hat{X}_{t,i,j}^{(k)} - X_{t,i,j}^{(k)})^2}, \quad (4.1)$$

with MAE defined analogously.

CRPS. The continuous ranked probability score evaluates the full predictive distribution of the small per-hour ensemble against the observation; lower is better. It rewards a forecast that is both sharp and calibrated, and is our primary probabilistic score.

Categorical precipitation scores. For threshold τ (mm h^{-1}), classify each pixel as wet ($\geq \tau$) or dry. With hits H , misses M , false alarms F :

$$\text{CSI}(\tau) = \frac{H}{H + M + F}, \quad \text{FAR}(\tau) = \frac{F}{H + F}, \quad \text{HSS}(\tau) = \frac{2(H \text{CN} - M F)}{(H + M)(M + \text{CN}) + (H + F)(F + \text{CN})}, \quad (4.2)$$

where CN is the correct-negative count. We report $\tau \in \{0.1, 1.0, 5.0, 10.0, 16.0\} \text{ mm h}^{-1}$; “-M” suffixes denote the mean over thresholds and “-P16” the $\tau = 16 \text{ mm h}^{-1}$ heavy-rain point. CSI and HSS reward correct detection (higher is better); FAR penalises over-

prediction (lower is better).



Fractions Skill Score. With neighbourhood radius n ,

$$\text{FSS}(\tau, n) = 1 - \frac{\sum_{i,j} (p_{i,j}^{\text{fc}} - p_{i,j}^{\text{ob}})^2}{\sum_{i,j} (p_{i,j}^{\text{fc}})^2 + \sum_{i,j} (p_{i,j}^{\text{ob}})^2}, \quad (4.3)$$

where $p_{i,j}$ is the fraction of τ -exceeding pixels in an $n \times n$ box centred at (i, j) . FSS mitigates the double-penalty problem and is reported at the heavy-rain point $\tau = 16 \text{ mm h}^{-1}$ for $n \in \{3, 7, 15\}$.

Radial log-PSD. We compute the 2D FFT of each qpepre field, square its magnitude, radially average to obtain $P(\kappa)$, and compare $\log P(\kappa)$ of forecast and observation. A model that blurs precipitation has a log-PSD that decays too fast at high wavenumbers.

Rollout metrics. Per-channel RMSE is additionally reported as a function of lead time $\ell \in \{1, \dots, 12\} \text{ h}$ to expose autoregressive error growth.

4.3 FlowCast versus Diffusion: Headline Comparison

Table 4.1 is the central result of the thesis: the EDM diffusion baseline and FlowCast (at three Euler-step counts) on the 2022 validation year, all at the matched $\approx 2 \text{ M}$ -sample budget, sharing the same frozen regression mean and cleaned loglp dataset. Figure 4.1 visualises the same scoreboard.

Read. At $K = 10$ FlowCast runs in 8.37 s per sequence versus 24.45 s for the ≈ 25 -NFE EDM Heun sampler — a **2.9 $\times$ speedup** — and at that budget it matches or beats



Table 4.1: Headline comparison on the 2022 validation year (cleaned 192×96 , $\log_{10}$ qppepre, ≈ 2 M training samples). Precipitation scores are in mm h^{-1} after denormalisation. “-M” = mean over thresholds, “-P16” = 16 mm h^{-1} . $\uparrow/\downarrow$ indicate the better direction; **bold** marks the best in each column. Time/Seq. is the wall-clock cost of one rollout sequence.

Method	Time/Seq. (s)	CRPS $\downarrow$	CSI-M $\uparrow$	FSS-P16 $\uparrow$	HSS-M $\uparrow$	FAR-M $\downarrow$	RMSE u10 $\downarrow$	RMSE v10 $\downarrow$	RM qppepre
EDM diffusion (≈ 25 NFE)	24.45	0.6895	0.1479	0.0009	0.1792	0.6958	1.6142	2.0709	0.9
FlowCast ($K=10$)	8.37	0.6886	0.1324	0.0068	0.1683	0.4280	1.5321	1.6655	0.9
FlowCast ($K=15$)	10.76	0.6712	0.1376	0.0058	0.1743	0.5444	1.5598	1.7002	0.9
FlowCast ($K=20$)	13.79	0.6807	0.1385	0.0005	0.1721	0.6869	1.5661	1.7893	0.9

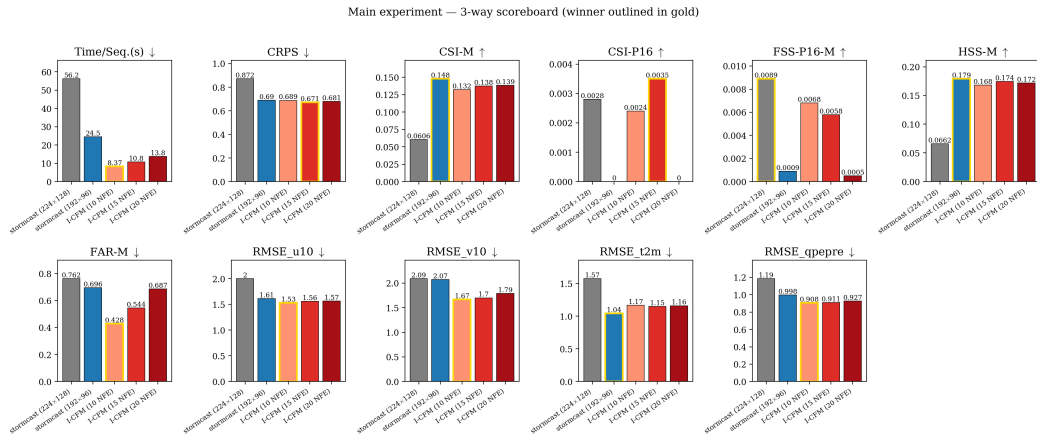


Figure 4.1: Per-metric comparison of the EDM diffusion baseline and FlowCast at $K \in \{10, 15, 20\}$ on the 2022 validation year. FlowCast attains its skill at roughly one-third of the diffusion wall-clock cost.

the diffusion baseline on most columns: CRPS is tied (0.6886 vs 0.6895), and FlowCast is clearly better on u10 RMSE (-5%), v10 RMSE (-20%), qpepre RMSE (-9%), heavy-rain FSS, and especially the false-alarm ratio (FAR-M 0.43 vs 0.70). The diffusion baseline retains an edge on the hit-based categorical scores CSI-M and HSS-M and on t2m RMSE (diffusion 1.0425 vs FlowCast 1.1669; the one channel where FlowCast trails, omitted from Table 4.1 for width but shown in Figure 4.2 (a)). Pushing FlowCast to $K = 15$ buys the best CRPS in the table (0.6712) at 10.76 s, still less than half the diffusion cost; beyond that, $K = 20$ shows no further gain and a small FAR regression, consistent with the straight-line ODE being essentially converged by ≈ 10 –15 steps.

Interpretation. The categorical picture is a coherent trade rather than a wash: FlowCast issues markedly fewer false alarms (low FAR) but also slightly fewer hits (lower CSI), i.e. it is the more conservative precipitation forecaster, while the diffusion baseline is more aggressive in placing wet pixels and is rewarded by CSI/HSS but penalised by FAR. Which is preferable is an operational choice; the unambiguous wins for FlowCast are the wind fields, precipitation RMSE, calibrated CRPS, and a $\sim 3\times$ lower inference cost.

Per-channel RMSE and CRPS. Figures 4.2 (a) and 4.2 (b) break the deterministic and probabilistic errors out by channel. FlowCast’s advantage is largest and consistent across both metrics on the meridional wind v10 and on the precipitation channel. On u10 the two metrics disagree — FlowCast has the lower RMSE but the diffusion baseline the lower CRPS — and on t2m the diffusion baseline leads on both, FlowCast’s temperature residual being marginally noisier at this budget.

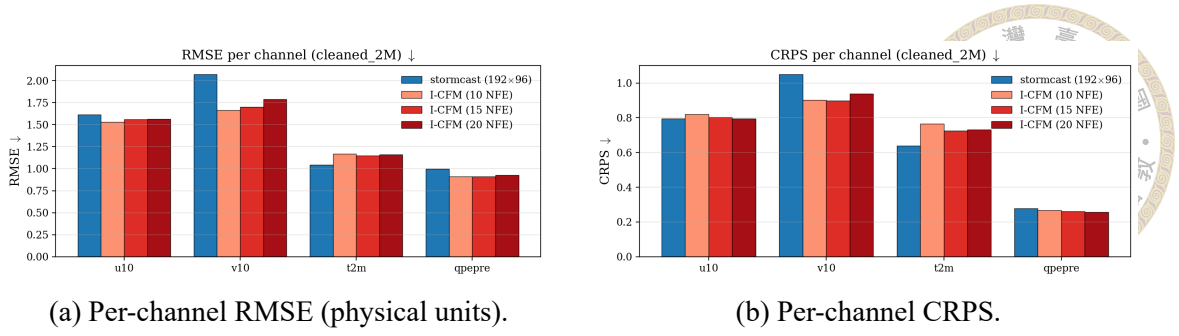


Figure 4.2: Per-channel deterministic (RMSE) and probabilistic (CRPS) errors for the diffusion baseline and FlowCast on the 2022 validation year. FlowCast wins on the winds and precipitation; the diffusion baseline leads on $t2m$.

4.4 Inference Cost: NFE and Wall-Clock Trade-off

The defining property of a straight-line flow is that it integrates accurately in very few steps. The three FlowCast rows of Table 4.1 already show that quality is essentially flat across $K \in \{10, 15, 20\}$ while wall-clock rises linearly, so the operational sweet spot sits at the low end of that range. Per NFE at the $K = 10$ operating point, FlowCast costs ≈ 0.84 s per network evaluation (single forward pass); because a fixed per-sequence overhead is amortised over more steps, the effective per-NFE cost actually falls toward ≈ 0.69 s at $K = 20$. Either way it undercuts the diffusion Heun sampler’s ≈ 0.98 s per evaluation (two evaluations per interior step), so FlowCast is cheaper both because it uses fewer steps and because each step is a single forward pass with no second-order correction.

The practical consequence is the one that motivated the thesis (Section 1.4). A 50-member, 24-hour ensemble rollout costs, per forecast cycle, 50×24 sequence-steps; at ≈ 0.84 s per NFE and $K = 10$, FlowCast brings the generative head’s contribution down by the same $\approx 3\times$ factor seen in the single-sequence timing, moving a configuration that the 25-NFE diffusion sampler places outside the between-radar-volume budget back inside it. Crucially, this speedup is obtained without a second distillation training stage: it is intrinsic to the straight-line model.



4.5 Precipitation Skill by Threshold

Mean categorical scores hide the threshold dependence that matters operationally.

Figure 4.3 plots CSI, FAR, and HSS against the rainfall threshold, and Figure 4.4 the heavy-rain FSS at three neighbourhood sizes.

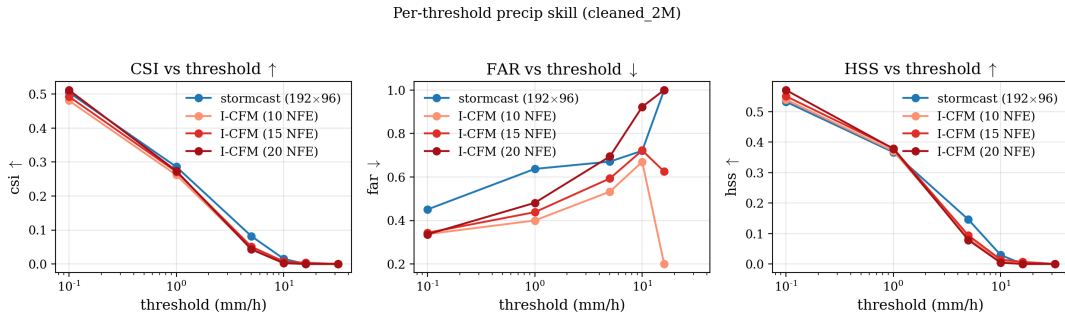


Figure 4.3: CSI, FAR, and HSS as a function of rainfall threshold for the diffusion baseline and FlowCast at $K \in \{10, 15, 20\}$. FlowCast holds a clear FAR advantage at every threshold; the diffusion baseline pulls ahead on CSI at the heavier 5–10 mm h⁻¹ thresholds.

Light to moderate rain. At $\tau = 0.1$ mm h⁻¹ the two heads are close on CSI (diffusion 0.505, FlowCast- K 20 0.512, FlowCast- K 10 0.481), but FlowCast’s false-alarm ratio is substantially lower (0.34 vs 0.45). The same ordering holds at $\tau = 1.0$ mm h⁻¹. This is the quantitative form of the “more conservative” behaviour noted above: FlowCast does not paint speculative light rain across dry pixels.

Heavy rain. At $\tau = 5.0$ and 10.0 mm h⁻¹ the diffusion baseline leads on CSI (0.082 vs 0.046 at 5 mm h⁻¹), reflecting its more aggressive placement of intense cells. FlowCast’s heavy-rain hit rate at this ≈ 2 M-sample budget is its weakest point and a clear target for the longer-training and tail-loss experiments listed in the project to-do. Interestingly, FlowCast’s neighbourhood FSS at $\tau = 16$ mm h⁻¹ (Figure 4.4) is higher than the diffusion baseline’s at $K = 10$: when a near-miss is scored against a spatial neighbourhood rather

than pixel-exactly, FlowCast’s heavy-rain structure is competitive even though its pixel-exact CSI is lower — evidence that the gap is partly a displacement/double-penalty artefact rather than a pure miss.

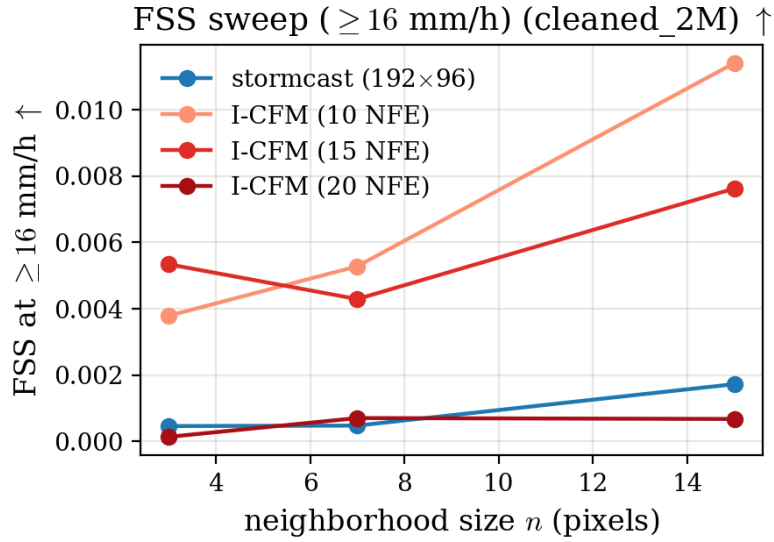
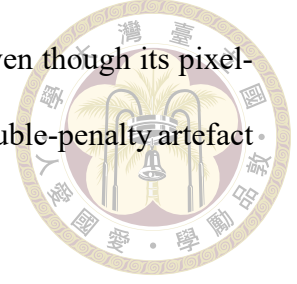


Figure 4.4: FSS at $\tau = 16 \text{ mm h}^{-1}$ for neighbourhood sizes $n \in \{3, 7, 15\}$. FlowCast at $K = 10$ and $K = 15$ exceeds the diffusion baseline once a spatial neighbourhood is allowed, indicating the heavy-rain deficit in pixel-exact CSI is partly a displacement penalty.

4.6 Autoregressive Rollout

Single-step skill understates operational risk because errors compound under autoregression. Figure 4.5 plots per-channel RMSE against lead time out to 12 hours.

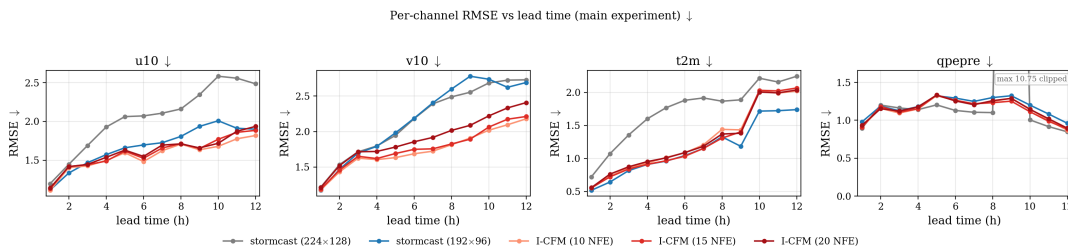
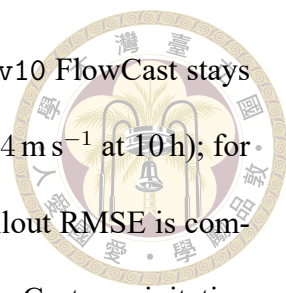


Figure 4.5: Per-channel RMSE as a function of autoregressive lead time (1–12 h) for the legacy baseline, the EDM diffusion baseline, and FlowCast. FlowCast tracks or improves on the diffusion baseline for the winds and precipitation across the horizon; its t_{2m} error grows somewhat faster at long lead times.



Read. Both heads show the expected monotone error growth. For v10 FlowCast stays below the diffusion baseline at every lead time (e.g. 2.02 m s^{-1} vs 2.74 m s^{-1} at 10 h); for u10 the two are close, with FlowCast slightly ahead. The qppe rollout RMSE is comparable between heads and does not diverge — the conservative FlowCast precipitation does not destabilise under rollout. The one adverse signal is t2m: FlowCast’s temperature error grows faster after $\approx 8 \text{ h}$ (reaching $\approx 2.0 \text{ K}$ at 10–12 h versus $\approx 1.7 \text{ K}$ for diffusion), consistent with its weaker single-step t2m number. No run exhibited the unphysical blow-up that aggressive few-step diffusion samplers are prone to, which we attribute to the straight-line ODE’s stability.

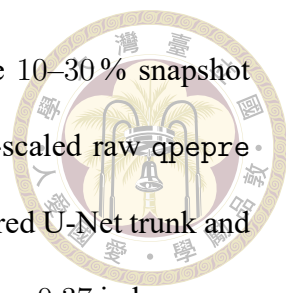
4.7 Ablation: log1p Precipitation Encoding

The single most consequential preprocessing decision is whether to store qppe as raw mm h^{-1} or as $\log(1 + x)$ before standardisation. Table 4.2 reports the within-architecture A/B test (single-step standardised validation RMSE at $\approx 2 \text{ M}$ samples) for both heads.

Table 4.2: Single-step standardised validation RMSE at $\approx 2 \text{ M}$ samples, log1p versus raw-mm/h qppe encoding. Values are dimensionless standard scores. **Bold** = better within each architecture pair on the non-qppe channels. The qppe column is not directly comparable across the encoding boundary (different units in standardised space); see the note.

Run	RMSE t2m	RMSE u10	RMSE v10	RMSE qppe [†]
diffusion, log1p	0.0994	0.3523	0.2762	0.7865
diffusion, raw mm/h	0.1245	0.3955	0.3115	0.4156
FlowCast, log1p	0.0835	0.2931	0.2463	0.3851
FlowCast, raw mm/h	0.1118	0.3079	0.2331	0.3471

Read. The log1p encoding improves both heads on t2m and u10 ($\sim 20\text{--}25\%$ on t2m, $\sim 5\text{--}11\%$ on u10) and improves the diffusion baseline’s v10 as well; the lone exception



is FlowCast’s v10, where the two encodings differ by less than the 10–30 % snapshot noise and so are effectively tied. The mechanism is that the badly-scaled raw qpepre channel (standard deviation 9.12 mm h^{-1}) propagates through the shared U-Net trunk and degrades the other channels; compressing it to a standard deviation of ≈ 0.37 in log space removes that pressure. The net benefit is similar for both architectures, so the encoding choice is essentially independent of the head, and we adopt log1p as canonical throughout. The qpepre column ($\dagger$) cannot be compared directly across the encoding boundary: in standardised space the two encodings carry different units, and inverting log1p on a scalar RMSE is invalid because $\exp m1$ does not commute with the root-mean-square. The mm h^{-1} -correct comparison is the denormalised scoreboard of Section 4.3, where `compare_diffusion_vs_flowcast.py` inverts the encoding per pixel before scoring.

4.8 Ablation: Precipitation Channel Weight

The precipitation weight w_{pr} (the last entry of `channel_weights`) trades precipitation skill against wind/temperature skill. Table 4.3 reports the full eight-point FlowCast sweep $w_{\text{pr}} \in [1.0, 2.4]$ (single-step standardised validation RMSE, log1p, $\approx 2 \text{ M}$ samples). Because `channel_weights` is applied only inside the training loss and not at evaluation, these RMSEs are directly comparable across rows without rescaling by w_{pr} .

Read. There is no clean monotone trend: both $w_{\text{pr}} = 1.4$ and $w_{\text{pr}} = 2.0$ beat their neighbours, while $w_{\text{pr}} = 1.6$ and $w_{\text{pr}} \geq 2.2$ are poor across the board. $w_{\text{pr}} = 1.4$ is the per-channel sweet spot for the wind fields (lowest u10 and v10 RMSE in the sweep), whereas $w_{\text{pr}} = 2.0$ is best on t2m; on qpepre the minimum is a near-tie among $w_{\text{pr}} \in \{1.2, 1.4, 2.0\}$ that sits inside the 10–30 % single-snapshot noise, so we do not read fine ordering between

Table 4.3: FlowCast precipitation-weight sweep, single-step standardised validation RMSE (log1p, ≈ 2 M samples). **Bold** = column minimum. $w_{\text{pr}} = 2.0$ is the headline run.

w_{pr}	RMSE t2m	RMSE u10	RMSE v10	RMSE qppepre
1.0	0.0994	0.3188	0.2326	0.5464
1.2	0.1032	0.2912	0.2110	0.3771
1.4	0.0890	0.2733	0.2050	0.3996
1.6	0.0930	0.3282	0.2391	0.6796
1.8	0.0983	0.2838	0.2075	0.4831
2.0 (headline)	0.0835	0.2931	0.2463	0.3851
2.2	0.0990	0.3178	0.2499	0.5178
2.4	0.0991	0.3075	0.2355	0.5515

those three. The robust conclusions are (i) some up-weighting of qppepre helps but the floor-to-ceiling spread is large and real (e.g. $w_{\text{pr}} = 1.4$ at 0.3996 vs $w_{\text{pr}} = 1.6$ at 0.6796 is a $\sim 70\%$ jump on qppepre RMSE); (ii) $w_{\text{pr}} \approx 2.0$ is a reasonable all-round default that wins t2m and is competitive on qppepre, which is why the canonical run adopts it; and (iii) the wind/precipitation trade-off is genuine and would justify dropping to $w_{\text{pr}} = 1.4$ if the winds were the priority.

4.9 Comparison Against the Legacy Baseline

To place both cleaned-pipeline heads in context, Table 4.4 adds the legacy old-StormCast EDM baseline (the original 224×128 raw-qppepre model on the legacy dataset). Because the legacy grid and encoding differ, this is a pipeline-level rather than a controlled head-to-head comparison, and the absolute magnitudes are not directly comparable to the cleaned rows (the field of view differs by the centre crop).

Read. The cleaning + log1p + retraining pipeline is a large step up over the legacy baseline on every metric (CRPS $0.87 \rightarrow 0.69$, CSI-M $0.06 \rightarrow 0.15$), independent of the generative head; on top of that, FlowCast adds the inference-cost and false-alarm wins at

Table 4.4: Legacy old-StormCast EDM baseline versus the cleaned-pipeline diffusion and FlowCast heads. Precipitation in mm h^{-1} . The legacy leg uses a different grid/encoding, so absolute values are comparable only in magnitude (see Section 3.2).

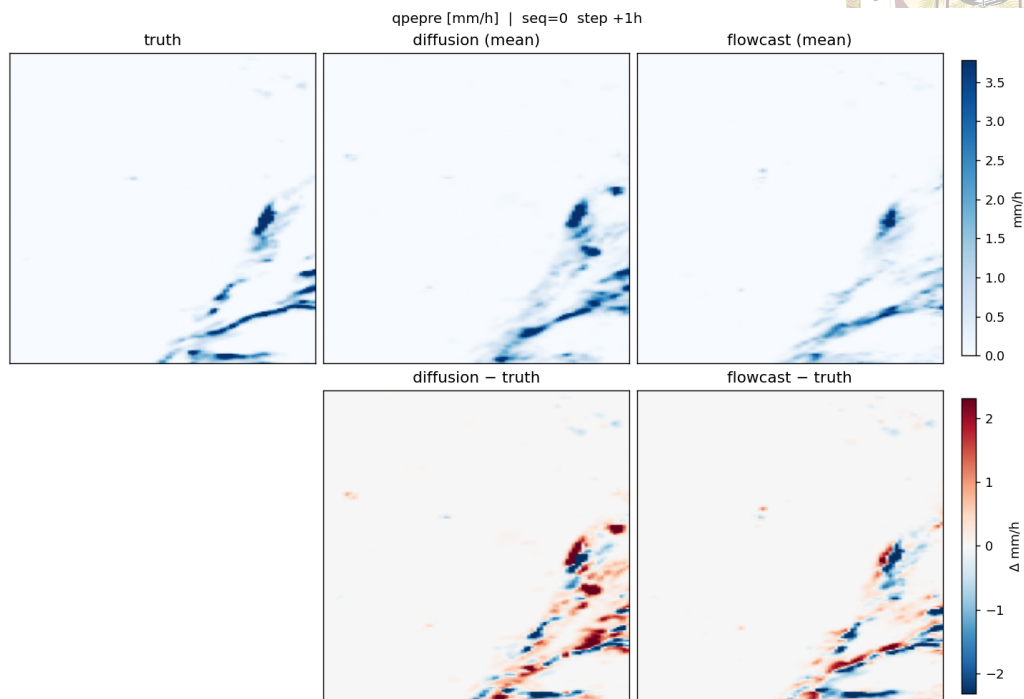
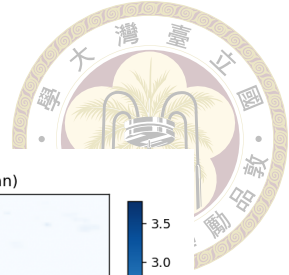
Method	Time/Seq. (s)	CRPS↓	CSI-M↑	FAR-M↓	RMSE qpepre↓
Legacy EDM (raw, 224×128)	56.20	0.8717	0.0606	0.7621	1.1852
Cleaned EDM (≈ 25 NFE)	24.45	0.6895	0.1479	0.6958	0.9983
Cleaned FlowCast ($K=10$)	8.37	0.6886	0.1324	0.4280	0.9084

parity on CRPS. The legacy-to-cleaned and cleaned-EDM-to-FlowCast gaps are therefore largely orthogonal: most of the headline improvement is the pipeline, and FlowCast’s specific contribution is to deliver comparable-or-better skill at a fraction of the cost without a teacher.

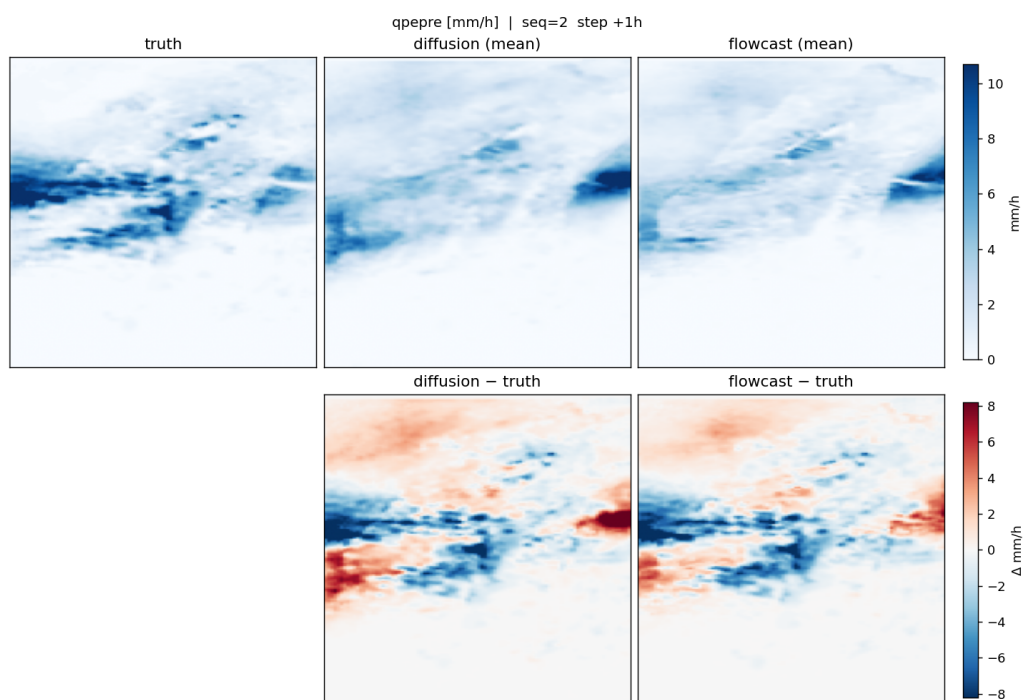
4.10 Qualitative Case Studies

Figure 4.6 shows side-by-side denormalised precipitation panels (truth, diffusion, FlowCast) for representative 2022 validation hours, and Figure 4.7 the same for the zonal wind. The qualitative reading matches the metrics: FlowCast’s precipitation is sharp and spatially coherent, with fewer speculative light-rain pixels in nominally dry regions (the FAR advantage made visible), while the diffusion baseline paints more widespread weak precipitation. On the wind field FlowCast’s structure is visibly closer to the RWRf target.

Spectral sharpness. Figure 4.8 (b) shows a representative FlowCast qpepre radial log-PSD against the target during training: the spectral-regularised FlowCast tracks the observation spectrum into the high-wavenumber band rather than rolling off early, which is the spectral signature of a sharp (non-blurry) generative forecast.



(a) Validation sequence A.



(b) Validation sequence B.

Figure 4.6: Denormalised hourly precipitation (mm h^{-1}): RWRf/QPEPRE truth, EDM diffusion, and FlowCast, for two 2022 validation sequences. FlowCast keeps nominally dry regions dry while preserving convective structure.

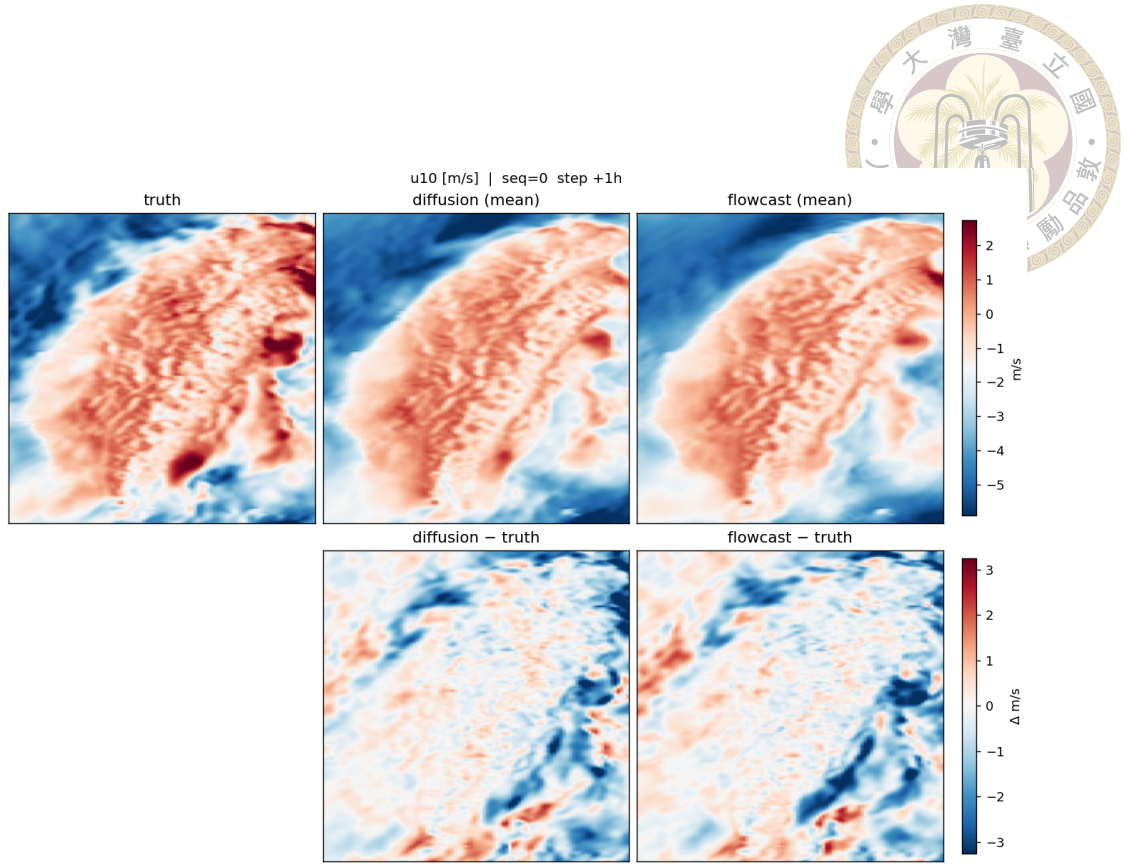


Figure 4.7: Denormalised 10 m zonal wind u_{10} (m s^{-1}): truth, diffusion, and FlowCast. FlowCast's field is closer to the RWRF target, consistent with its lower wind RMSE.

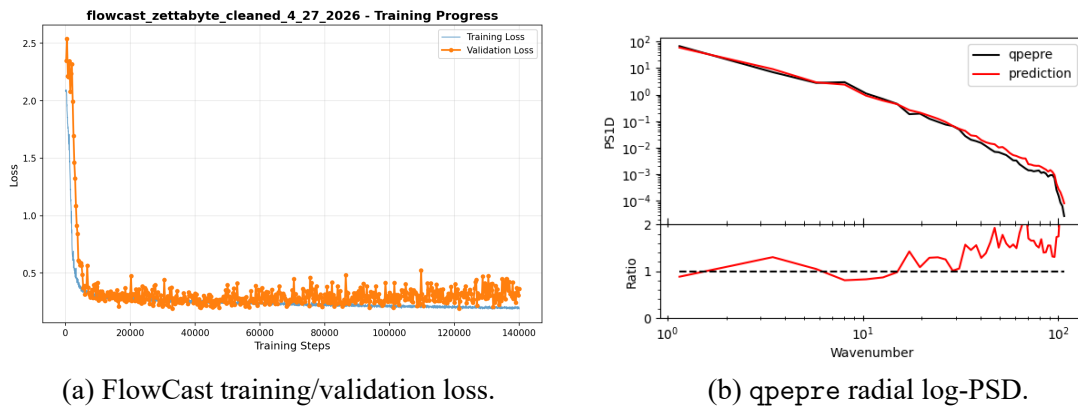


Figure 4.8: (a) FlowCast loss curves over training; (b) radial log power-spectral density of a validation qpepre sample versus the target. The spectral-matching regulariser keeps the predicted spectrum from decaying too fast at high wavenumbers.

4.11 Failure Modes



We document three failure modes observed during development.

Heavy-rain under-detection. FlowCast’s weakest metric is pixel-exact CSI at the heavier thresholds ($5\text{--}10\text{ mm h}^{-1}$): its conservative, low-false-alarm behaviour costs hits on intense cells. Diagnosis: the per-threshold CSI curve (Figure 4.3) crossing below the diffusion baseline above 5 mm h^{-1} , partly recovered by the neighbourhood FSS. Mitigation candidates (deferred to future work): longer training, a higher w_{pr} paired with the spectral term, or an explicit tail-quantile loss on `qpepre`.

t2m rollout drift. FlowCast’s temperature error grows faster than the diffusion baseline’s after ≈ 8 rollout hours (Figure 4.5). This is the autoregressive signature of its weaker single-step `t2m` number. Mitigation: a per-channel weight that does not starve `t2m`, or a short teacher-forcing warmup.

EMA dependence. Loading the online `checkpoint_*.pt` instead of `ema_state.pt` produces visibly noisier FlowCast samples. This is not a model failure but an evaluation pitfall; all reported FlowCast numbers use the EMA shadow, and the evaluation scripts default to it.

4.12 Discussion

The headline question of the thesis is whether a single-run, teacher-free flow-matching residual can replace the EDM diffusion residual in a regional convection-permitting fore-

caster without losing the features that matter for Taiwan precipitation — and do so cheaply enough to deploy. The evidence at the matched ≈ 2 M-sample budget is a qualified yes: at $K = 10$ Euler steps FlowCast matches the diffusion baseline’s CRPS, beats it on wind and precipitation RMSE, cuts its false-alarm ratio by $\approx 38\%$ (FAR-M 0.43 vs 0.70), and runs $\approx 2.9\times$ faster, while trading away some pixel-exact heavy-rain hit rate and some t2m accuracy.

Three caveats bound these conclusions. First, all numbers are at a single ≈ 2 M-sample snapshot with 10–30 % validation noise; the FlowCast canonical run continues to ≈ 13.4 M samples (step 140,000) and a final comparison at converged budgets is part of the remaining work (Chapter 5 and the project to-do). Second, both heads are anchored to the same frozen regression mean and the same cleaned log1p dataset, so the comparison is clean but specific to this Taiwan RWRF setup; transfer to other domains is untested. Third, the precipitation-encoding gotcha of Section 4.7 means the regression mean and residual head must always share the log1p encoding — mixing encodings silently corrupts the residual distribution. The next chapter summarises what we have learned and lays out the concrete experiments and data still needed to turn this provisional result into a defensible thesis claim.



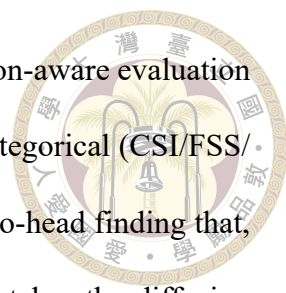


Chapter 5 Conclusion and Future Work

5.1 Summary of Contributions

This thesis asked whether a single-run, teacher-free flow-matching residual can replace the EDM diffusion residual in a regional, convection-permitting StormCast forecaster over the Taiwan domain — delivering the speed a curved diffusion ODE cannot, without sacrificing the sharpness and probabilistic calibration that motivated generative forecasting in the first place. Starting from the StormCast two-stage decomposition (a frozen deterministic regression mean plus a generative residual head) trained on the Taiwan RWRF dataset, we adapted Conditional Flow Matching into **FlowCast**, a residual head that learns a straight-line noise-to-data velocity field and samples it with a handful of explicit Euler steps, and we compared it head-to-head against the EDM diffusion residual on the full 2022 hourly validation year.

The concrete contributions are: (i) a single-training-run FlowCast residual pipeline, anchored to exactly the same frozen regression mean, backbone, and σ_{data} as the diffusion baseline so that the comparison isolates the generative trajectory; (ii) a precipitation-aware loss — channel-weighted L2 plus a radial log-PSD regulariser on q_{pre} — together with



log1p-encoding and precipitation-weight ablations; (iii) a precipitation-aware evaluation suite spanning deterministic (RMSE/MAE), probabilistic (CRPS), categorical (CSI/FSS/HSS/FAR), spectral, and rollout metrics; and (iv) the resulting head-to-head finding that, at a matched ≈ 2 M-sample budget, FlowCast at 10 Euler steps matches the diffusion baseline’s CRPS, beats it on wind and precipitation RMSE and on false-alarm ratio, and runs $\approx 2.9\times$ faster, while conceding some pixel-exact heavy-rain hit rate and some t2m accuracy.

5.2 Limitations

Several limitations should be kept in mind when interpreting these results.

Single training snapshot. The headline comparison is taken at a single ≈ 2 M-sample checkpoint, where single-snapshot validation noise is 10–30 %. The canonical FlowCast run continues to ≈ 13.4 M samples, and the diffusion baseline likewise has more training available; a converged-budget comparison is not yet in hand.

Single regression mean and single domain. Both heads are anchored to one frozen regression mean trained on one regional domain (Taiwan RWRF). We cannot assess transfer to other convection-permitting domains (HRRR/CONUS StormCast) without retraining, nor disentangle how much of FlowCast’s behaviour depends on the specific quality of this regression mean.

Heavy-rain detection. FlowCast’s conservative, low-false-alarm behaviour costs pixel-exact hits at the heavier rainfall thresholds. The neighbourhood FSS recovers part of this,

suggesting a displacement rather than a pure miss, but the deterministic heavy-rain CSI is a genuine weakness at the current budget.



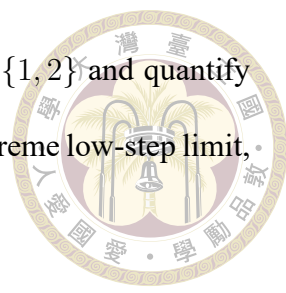
Limited probabilistic evaluation. CRPS is computed from a small per-hour ensemble. A large-ensemble calibration study (rank histograms, spread–skill) — which a cheap few-step sampler makes affordable — has not been run.

No few-step diffusion comparator. We compare FlowCast against the full ≈ 25 -NFE diffusion sampler, not against a distilled few-step diffusion student. Whether a distilled diffusion model could close FlowCast’s speed gap while keeping the diffusion baseline’s heavy-rain CSI is an open question this thesis does not answer.

5.3 Future Work

The most promising directions, several of which are scoped concretely in the accompanying project to-do, are:

1. **Train to convergence and re-score.** Run both heads to a matched, converged sample budget and repeat the full metric suite, smoothing over several validation snapshots to beat down the 10–30 % noise. This is the single most important step to turn the provisional headline into a defensible claim.
2. **Close the heavy-rain gap.** Test an explicit tail-quantile or focal loss on q_{pepre} , a higher w_{pr} paired with the spectral term, and a stronger $\text{asinh}/\log 1p$ dynamic-range transform, targeting the 5–10 mm h^{−1} CSI deficit without inflating false alarms.

- 
3. **One- and two-step FlowCast.** Push the NFE sweep to $K \in \{1, 2\}$ and quantify exactly how much quality the straight-line path retains at the extreme low-step limit, where the operational payoff is largest.
 4. **Stochastic-interpolant / bridge variants.** Generalise the straight-line flow to an anchored stochastic-interpolant bridge that starts from the regression mean rather than white noise, which may shorten the effective transport distance and further sharpen precipitation.
 5. **Large-ensemble calibration.** Exploit FlowCast's low per-sample cost to run 50–100-member ensembles and report CRPS, rank histograms, and spread–skill — the uncertainty-quantification payoff that cheap sampling unlocks.
 6. **Transfer to other domains.** Apply the same single-run FlowCast recipe to another convection-permitting regression mean (e.g. an HRRR-trained StormCast) to measure how much of the pipeline is Taiwan-specific.

5.4 Closing Remarks

Generative modelling is almost certainly the right inductive bias for short-range convective-scale forecasting; the obstacle to deploying it has been the multi-step diffusion sampler. This thesis shows that, on a real regional dataset, a straight-line flow-matching residual can recover most of the diffusion baseline's skill — and exceed it on winds, precipitation RMSE, and false alarms — at roughly a third of the inference cost, trained in a single run with no teacher. The remaining gaps (heavy-rain hits, t_{2m} drift, convergence at a larger budget) are concrete and addressable. Making sharp, probabilistic, regional convective forecasting fast enough to run between radar volumes is within reach, and a single-stage

flow-matching residual is a credible path to it.







References

- [1] M. Andrychowicz, L. Espeholt, D. Li, S. Merchant, A. Merose, F. Zyda, S. Agrawal, and N. Kalchbrenner. Deep learning for day forecasts from sparse observations. arXiv preprint arXiv:2306.06079, 2023.
- [2] K. Bi, L. Xie, H. Zhang, X. Chen, X. Gu, and Q. Tian. Accurate medium-range global weather forecasting with 3D neural networks. Nature, 619:533–538, 2023.
- [3] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. In Advances in Neural Information Processing Systems (NeurIPS), 2018.
- [4] L. Espeholt, S. Agrawal, C. K. Sønderby, M. Kumar, J. Heek, C. Bromberg, C. Gazeau, R. Carver, M. Andrychowicz, J. Hickey, A. Bell, and N. Kalchbrenner. Deep learning for twelve hour precipitation forecasts. Nature Communications, 13, 2022.
- [5] H. Hersbach, B. Bell, P. Berrisford, S. Hirahara, A. Horányi, J. Muñoz-Sabater, J. Nicolas, C. Peubey, R. Radu, D. Schepers, A. Simmons, C. Soci, S. Abdalla, X. Abellan, G. Balsamo, P. Bechtold, G. Biavati, J. Bidlot, M. Bonavita, G. D. Chiara, P. Dahlgren, D. Dee, M. Diamantakis, R. Dragani, J. Flemming, R. Forbes, M. Fuentes, A. Geer, L. Haimberger, S. Healy, R. J. Hogan, E. Hólm, M. Janisková,

S. Keeley, P. Laloyaux, P. Lopez, C. Lupu, G. Radnoti, P. de Rosnay, I. Rozum, F. Vamborg, S. Villaume, and J.-N. Thépaut. The ERA5 global reanalysis. Quarterly Journal of the Royal Meteorological Society, 146(730):1999–2049, 2020.

[6] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In Advances in Neural Information Processing Systems (NeurIPS), 2020.

[7] J. Ho and T. Salimans. Classifier-free diffusion guidance. arXiv preprint arXiv:2207.12598, 2022.

[8] T. Karras, M. Aittala, T. Aila, and S. Laine. Elucidating the design space of diffusion-based generative models. In Advances in Neural Information Processing Systems (NeurIPS), 2022.

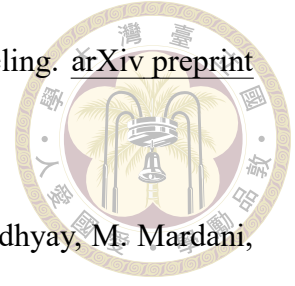
[9] R. Lam, A. Sanchez-Gonzalez, M. Willson, P. Wirsberger, M. Fortunato, F. Alet, S. Ravuri, T. Ewalds, Z. Eaton-Rosen, W. Hu, A. Meroze, S. Hoyer, G. Holland, O. Vinyals, J. Stott, A. Pritzel, S. Mohamed, and P. Battaglia. Learning skillful medium-range global weather forecasting. Science, 382(6677):1416–1421, 2023.

[10] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In International Conference on Learning Representations (ICLR), 2023.

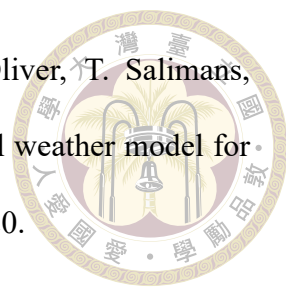
[11] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In International Conference on Learning Representations (ICLR), 2023.

[12] J. Pathak, Y. Cohen, P. Garg, P. Harrington, N. Brenowitz, D. Durran, M. Mardani, A. Vahdat, S. Xu, K. Kashinath, and M. Pritchard. Kilometer-scale convec-

tion allowing model emulation using generative diffusion modeling. [arXiv preprint arXiv:2408.10958](#), 2024.



- [13] J. Pathak, S. Subramanian, P. Harrington, S. Raja, A. Chattopadhyay, M. Mardani, T. Kurth, D. Hall, Z. Li, K. Azizzadenesheli, P. Hassanzadeh, K. Kashinath, and A. Anandkumar. FourCastNet: A global data-driven high-resolution weather model using adaptive Fourier neural operators. [arXiv preprint arXiv:2202.11214](#), 2022.
- [14] I. Price, A. Sanchez-Gonzalez, F. Alet, T. R. Andersson, A. El-Kadi, D. Masters, T. Ewalds, J. Stott, S. Mohamed, P. Battaglia, R. Lam, and M. Willson. Probabilistic weather forecasting with machine learning. [Nature](#), 2024.
- [15] S. Ravuri, K. Lenc, M. Willson, D. Kangin, R. Lam, P. Mirowski, M. Fitzsimons, M. Athanassiadou, S. Kashem, S. Madge, R. Prudden, A. Mandhane, A. Clark, A. Brock, K. Simonyan, R. Hadsell, N. Robinson, E. Clancy, A. Arribas, and S. Mohamed. Skilful precipitation nowcasting using deep generative models of radar. [Nature](#), 597:672–677, 2021.
- [16] B. P. Ribeiro and J. F. Pucer. FlowCast: Advancing precipitation nowcasting with conditional flow matching. [arXiv preprint](#), 2025.
- [17] N. M. Roberts and H. W. Lean. Scale-selective verification of rainfall accumulations from high-resolution forecasts of convective events. [Monthly Weather Review](#), 136(1):78–97, 2008.
- [18] T. Salimans and J. Ho. Progressive distillation for fast sampling of diffusion models. In [International Conference on Learning Representations \(ICLR\)](#), 2022.
- [19] A. Sauer, D. Lorenz, A. Blattmann, and R. Rombach. Adversarial diffusion distillation. [arXiv preprint arXiv:2311.17042](#), 2023.

- 
- [20] C. K. Sønderby, L. Espeholt, J. Heck, M. Dehghani, A. Oliver, T. Salimans, S. Agrawal, J. Hickey, and N. Kalchbrenner. MetNet: A neural weather model for precipitation forecasting. arXiv preprint arXiv:2003.12140, 2020.
- [21] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In International Conference on Learning Representations (ICLR), 2021.
- [22] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever. Consistency models. In International Conference on Machine Learning (ICML), 2023.
- [23] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. In International Conference on Learning Representations (ICLR), 2021.
- [24] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, G. Wolf, and Y. Bengio. Improving and generalizing flow-based generative models with mini-batch optimal transport. Transactions on Machine Learning Research, 2024.



Appendix A — Hyperparameters and Training Configuration

A.1 EDM Diffusion Baseline Hyperparameters

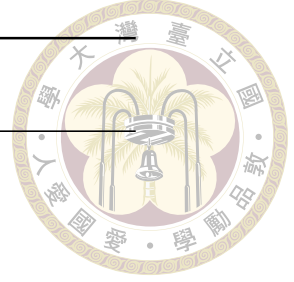
The EDM diffusion baseline uses the following parameters. FlowCast shares the data scale σ_{data} verbatim so that the standardised-residual statistics and per-channel loss scaling are identical across the two heads (Section 3.4):

Parameter	Value
σ_{min}	0.002
σ_{max}	80
σ_{data}	0.5
ρ	7
Sampler	Heun
N (steps)	≈ 13 (≈ 25 NFE)
Global batch size	64
Optimiser	AdamW

A.2 FlowCast (Conditional Flow Matching)



Hyperparameters copied from `stormcast/config/model/flowcast.yaml` and `stormcast/config/training/flowcast.yaml`:



Parameter	Value
σ_{data}	0.5
σ_{path} (I-CFM path std)	0.01
t_{ε} (time clamp)	10^{-5}
Time embedding scale τ	1000
diffusion_conditions	[state, regression, invariant]
regression_conditions	[state, background, invariant]
Backbone channel_mult	[1, 2, 2, 2, 2]
Optimiser	AdamW
Adam (β_1, β_2)	(0.9, 0.999)
Learning rate	5×10^{-4}
Weight decay	1×10^{-4}
Warmup steps (linear)	4,000
Min LR ratio (cosine floor)	10^{-2}
Total training steps	4×10^5
Global batch size	96
Gradient clip (global L2)	1.0
EMA decay (fixed)	0.999
Channel weights β	$(1, 1, 1, w_{\text{pr}})$, $w_{\text{pr}} = 2$ (headline)
Spectral weight λ_{spec}	0.1
Spectral channels	{qpepre}
Validation sampler	Euler, $K = 10$
Inference solvers	euler / midpoint



A.3 Ablation Grids

The two ablations of Chapter 4 vary a single knob each, holding the rest of the Flow-Cast config fixed:

Ablation	Values swept
Precipitation encoding (Section 4.7)	$\log_{10} p$ / raw mm h^{-1}
Precipitation weight w_{pr} (Section 4.8)	$\{1.0, 1.2, 1.4, 1.6, 1.8, 2.0, 2.2, 2.4\}$
Inference steps K (Section 4.4)	$\{1, 2, 4, 8, 10, 15, 20\}$

A.4 Per-Channel Dataset Statistics

High-resolution target statistics computed on the 2019-08 – 2021-12 training split (original store; the cleaned store re-derives statistics after the $\log_{10} p$ encoding of q_{pepre}):

Channel	Mean	Std
t2m (K)	295.98	5.61
u10 (m s^{-1})	−1.58	3.84
v10 (m s^{-1})	−2.65	5.66
qpepre (mm h^{-1} , raw)	0.182	9.12
qpepre ($\log(1 + x)$ space)	—	≈ 0.37



Appendix B — Reproducibility

B.1 Code Availability

The full training and evaluation code for this thesis lives in the Stormcast-Distillation repository. The key entry points are:

- `stormcast/train.py` — regression and EDM diffusion baseline training (Hydra config selects the mode).
- `stormcast/train_flowcast.py` — FlowCast (Conditional Flow Matching) training.
- `stormcast/utils/trainer_flowcast.py` — FlowCast training loop (`flowcast_training_loop` with EMA).
- `stormcast/utils/flowcast_loss.py` — FlowCastLoss (I-CFM + per-channel weights + radial log-PSD).
- `stormcast/utils/flowcast_precond.py` — FlowCastPrecond velocity wrapper.
- `stormcast/utils/nn.py` — `get_preconditioned_architecture`, `build_network_condition` and the samplers `diffusion_model_forward` / `flowcast_model_forward`.

- `experiment_scripts/compare_diffusion_vs_flowcast.py` → the head-to-head rollout scoreboard harness used for Chapter 4.



B.2 Dataset and Checkpoints

Zarr store. The cleaned Taiwan RWRF dataset used for the reported experiments is the local Zarr store

`exp_3_..._full_cleaned_4_27_2026/`

with sub-groups LowRes/, HighRes/, and invariants/, a 192×96 grid, and log1p-encoded qppepre (Section 3.2).

Checkpoints (cleaned, log1p, ≈ 2 M-sample anchor).

- Regression mean: `StormCastUNet.0.8000.mdlus` (shared by both heads).
- EDM diffusion baseline: `EDMPrecond.0.31000.mdlus`.
- FlowCast: `FlowCastPrecond.0.20000.mdlus` plus its `ema_state.pt` (inference uses the EMA shadow, never the online checkpoint). The canonical FlowCast run continues to `FlowCastPrecond.0.140000.mdlus`.

B.3 Commands

All commands below launch from the `stormcast/` directory. The shipped configs point at stale cluster paths and outdated `valid_dates`; override `dataset.location`, the

regression weights, and `dataset.valid_dates=['2022/01/01','2022/12/31']` on the local workstation.



Regression mean (pre-train once).

```
python train.py \  
  
    dataset.location=.../zarr_..._cleaned_4_27_2026 \  
  
    training.experiment_name=regression_main
```

EDM diffusion baseline.

```
python train.py --config-name=diffusion \  
  
    model.regression_weights=.../StormCastUNet.0.8000.mdlus \  
  
    dataset.location=.../zarr_..._cleaned_4_27_2026 \  
  
    training.experiment_name=diffusion_main
```

FlowCast.

```
python train_flowcast.py \  
  
    model.regression_weights=.../StormCastUNet.0.8000.mdlus \  
  
    dataset.location=.../zarr_..._cleaned_4_27_2026 \  
  
    training.experiment_name=flowcast_main
```

Auto-resume. Re-running the same command picks up from the latest checkpoint; Flow-Cast restores both `checkpoints_flowcast/checkpoint_latest.pt` and `ema_state.pt`.

Head-to-head evaluation. The scoreboard, per-threshold, FSS, CRPS, rollout, and qualitative-panel outputs of Chapter 4 are produced by



```
bash experiment_scripts/run_main_experiment.sh
```

which runs `compare_diffusion_vs_flowcast.py` and writes `scoreboard.{md,csv}`, `per_threshold.csv`, `fss_p16.csv`, `rmse_per_step.csv`, `crps_per_channel.csv`, and a `panels/` tree in physical units (qpepre in mm h^{-1} after inverse-log1p). FlowCast evaluation loads `ema_state.pt` by default.



Appendix C — Detailed Derivations

This appendix collects full line-by-line derivations of every identity used in Chapter 2 without passing through to the literature. The goal is that any reader with a working knowledge of multivariate Gaussians, Jensen’s inequality, and basic Itô calculus can reconstruct the tractable training objective of DDPM, DDIM, EDM, and Flow Matching from scratch. Notation in each section is local; all symbols have the same meaning as in Chapter 2 unless otherwise specified.

C.1 DDPM Forward-Process Marginals

The DDPM forward process is a Markov chain with Gaussian transitions

$$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I), \quad t = 1, \dots, T,$$

with $\{\beta_t\} \in (0, 1)$ a fixed variance schedule. Write $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. We claim

$$q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I). \quad (\text{C.1})$$

Proof by induction on t . For $t = 1$, Equation (C.1) is precisely the definition of $q(x_1 | x_0)$. Assume it holds for some $t \geq 1$. One step of the forward chain writes

$$x_{t+1} = \sqrt{\alpha_{t+1}} x_t + \sqrt{\beta_{t+1}} \epsilon_{t+1},$$

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \bar{\epsilon}_t,$$

with $\epsilon_{t+1} \sim \mathcal{N}(0, I)$ independent of $\bar{\epsilon}_t \sim \mathcal{N}(0, I)$ (the second line is the inductive hypothesis rewritten as a reparameterisation). Substituting,

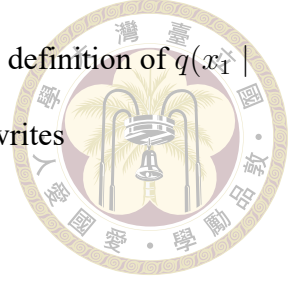
$$\begin{aligned} x_{t+1} &= \sqrt{\alpha_{t+1} \bar{\alpha}_t} x_0 + \sqrt{\alpha_{t+1} (1 - \bar{\alpha}_t)} \bar{\epsilon}_t + \sqrt{\beta_{t+1}} \epsilon_{t+1} \\ &= \sqrt{\bar{\alpha}_{t+1}} x_0 + \underbrace{\sqrt{\alpha_{t+1} (1 - \bar{\alpha}_t)} \bar{\epsilon}_t + \sqrt{1 - \alpha_{t+1}} \epsilon_{t+1}}_{=:\xi_{t+1}}. \end{aligned}$$

The term ξ_{t+1} is a sum of two independent zero-mean Gaussians, hence Gaussian, with variance

$$\text{Var}(\xi_{t+1}) = \alpha_{t+1} (1 - \bar{\alpha}_t) + (1 - \alpha_{t+1}) = 1 - \alpha_{t+1} \bar{\alpha}_t = 1 - \bar{\alpha}_{t+1}.$$

Therefore $x_{t+1} = \sqrt{\bar{\alpha}_{t+1}} x_0 + \sqrt{1 - \bar{\alpha}_{t+1}} \epsilon$ for some $\epsilon \sim \mathcal{N}(0, I)$, which is (C.1) at time $t + 1$. □

This closed form is the first tractability result: the marginals are known analytically at every time, so training can sample (x_t, x_0, ϵ) in one line rather than simulating the chain.





C.2 DDPM Posterior $q(x_{t-1} \mid x_t, x_0)$

Using Bayes' rule,

$$q(x_{t-1} \mid x_t, x_0) \propto q(x_t \mid x_{t-1}) q(x_{t-1} \mid x_0).$$

Both factors are Gaussian:

$$\log q(x_t \mid x_{t-1}) = -\frac{1}{2\beta_t} \|x_t - \sqrt{\alpha_t} x_{t-1}\|^2 + \text{const},$$

$$\log q(x_{t-1} \mid x_0) = -\frac{1}{2(1 - \bar{\alpha}_{t-1})} \|x_{t-1} - \sqrt{\bar{\alpha}_{t-1}} x_0\|^2 + \text{const}.$$

Adding and collecting terms in x_{t-1} produces a quadratic form $-\frac{1}{2\tilde{\beta}_t} \|x_{t-1} - \tilde{\mu}_t\|^2 + \text{const}$,

from which we can read off $\tilde{\beta}_t$ from the coefficient of $\|x_{t-1}\|^2$ and $\tilde{\mu}_t$ from the linear term.

Variance. The coefficient of $\|x_{t-1}\|^2$ is

$$\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} = \frac{\alpha_t(1 - \bar{\alpha}_{t-1}) + \beta_t}{\beta_t(1 - \bar{\alpha}_{t-1})} = \frac{1 - \bar{\alpha}_t}{\beta_t(1 - \bar{\alpha}_{t-1})},$$

so

$$\tilde{\beta}_t = \frac{\beta_t(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}. \quad (\text{C.2})$$

Mean. The linear-in- x_{t-1} term contributes

$$\frac{\sqrt{\alpha_t}}{\beta_t} x_t x_{t-1} + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} x_0 x_{t-1},$$

from which, equating with $\tilde{\mu}_t/\tilde{\beta}_t$,

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} x_0. \quad (\text{C.3})$$



Equations (C.2) and (C.3) recover Equation (2.4) of the main text.

C.3 DDPM Variational Bound (ELBO)

For any distribution $q(x_{1:T} | x_0)$, Jensen's inequality gives

$$-\log p_\theta(x_0) \leq \mathbb{E}_{q(x_{1:T}|x_0)} \left[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T} | x_0)} \right] = \mathbb{E}_q \left[\log \frac{q(x_{1:T} | x_0)}{p_\theta(x_{0:T})} \right].$$

Choosing $q(x_{1:T} | x_0)$ as the DDPM forward chain and $p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t)$ as the learned reverse chain, the ratio factors:

$$\log \frac{q(x_{1:T} | x_0)}{p_\theta(x_{0:T})} = -\log p(x_T) + \sum_{t=1}^T \log \frac{q(x_t | x_{t-1})}{p_\theta(x_{t-1} | x_t)}.$$

Using the Markov property $q(x_t | x_{t-1}) = q(x_t | x_{t-1}, x_0)$ and Bayes' rule,

$$q(x_t | x_{t-1}, x_0) = \frac{q(x_{t-1} | x_t, x_0) q(x_t | x_0)}{q(x_{t-1} | x_0)},$$

so for $t \geq 2$

$$\log \frac{q(x_t | x_{t-1})}{p_\theta(x_{t-1} | x_t)} = \log \frac{q(x_{t-1} | x_t, x_0)}{p_\theta(x_{t-1} | x_t)} + \log \frac{q(x_t | x_0)}{q(x_{t-1} | x_0)}.$$

Telescoping the second term across $t = 2, \dots, T$ gives $\log q(x_T | x_0) - \log q(x_1 | x_0)$.

Collecting all the pieces,

$$-\log p_\theta(x_0) \leq \mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(x_T | x_0) \| p(x_T))}_{L_T} + \sum_{t=2}^T \underbrace{D_{\text{KL}}(q(x_{t-1} | x_t, x_0) \| p_\theta(x_{t-1} | x_t))}_{L_{t-1}} - \underbrace{\log p_\theta(x_0 | x_1)}_{L_0} \right]$$

which is Equation (2.5) of the main text.

Two practical simplifications are available:

1. L_T contains no learnable parameter. Because $\bar{\alpha}_T \rightarrow 0$ for a well-designed schedule,

$$q(x_T | x_0) \rightarrow \mathcal{N}(0, I) = p(x_T) \text{ and } L_T \rightarrow 0.$$

2. L_0 is a discrete log-likelihood. (author?) [6] treat it as a separate decoder term,

which in practice is subsumed by the common ϵ -MSE loss at $t = 1$.

C.4 DDPM Simplified ϵ -Loss

We now collapse the middle-sum terms of the ELBO into an MSE.

KL between two Gaussians with matched covariance. For two Gaussians $p = \mathcal{N}(\mu_p, \Sigma)$

and $q = \mathcal{N}(\mu_q, \Sigma)$ with the same covariance,

$$D_{\text{KL}}(p \| q) = \frac{1}{2} (\mu_p - \mu_q)^\top \Sigma^{-1} (\mu_p - \mu_q). \quad (\text{C.4})$$

DDPM takes $p_\theta(x_{t-1} | x_t) = \mathcal{N}(\mu_\theta(x_t, t), \tilde{\beta}_t I)$, matching the posterior variance (C.2).

Then

$$L_{t-1} = \frac{1}{2\tilde{\beta}_t} \mathbb{E}_q [\|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2] + \text{const.}$$

Reparameterisation in ϵ . From $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$ we have $x_0 = (x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon)/\sqrt{\bar{\alpha}_t}$.

Substituting into Equation (C.3),

$$\begin{aligned}\tilde{\mu}_t(x_t, x_0) &= \frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \cdot \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon}{\sqrt{\bar{\alpha}_t}} \\ &= \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right),\end{aligned}$$

where the last simplification uses $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$ and collects the two coefficients of x_t . This motivates parameterising μ_θ through an ϵ -predictor $\epsilon_\theta(x_t, t)$ with the same functional form,

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right).$$

Substituting both expressions back into L_{t-1} ,

$$L_{t-1} = \frac{\beta_t^2}{2\tilde{\beta}_t \alpha_t (1 - \bar{\alpha}_t)} \mathbb{E}_{x_0, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2] + \text{const.}$$

(author?) [6] report that dropping the t -dependent prefactor and averaging over $t \sim \mathcal{U}(\{1, \dots, T\})$ gives a numerically better-behaved objective empirically, yielding Equation (2.8):

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2]. \quad (\text{C.5})$$

Why this is tractable in one forward pass. The entire training procedure, given a mini-batch $\{x_0^{(i)}\}$, is:

1. Sample $t \sim \mathcal{U}(\{1, \dots, T\})$ and $\epsilon \sim \mathcal{N}(0, I)$.
2. Form $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$.
3. Compute $\|\epsilon - \epsilon_\theta(x_t, t)\|^2$ with one network evaluation.

The chain is never simulated; neither the posterior nor the KL is materialised. This is the entire “tractability” story for DDPM.



C.5 Tweedie’s Formula: Score and Denoiser

Let $x_0 \sim p_{\text{data}}$ and $x = x_0 + \sigma \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$. Write $p_\sigma(x) = \int p_{\text{data}}(x_0) \mathcal{N}(x; x_0, \sigma^2 I) dx_0$.

Differentiating under the integral,

$$\nabla_x p_\sigma(x) = \int p_{\text{data}}(x_0) \nabla_x \mathcal{N}(x; x_0, \sigma^2 I) dx_0 = \int p_{\text{data}}(x_0) \frac{x_0 - x}{\sigma^2} \mathcal{N}(x; x_0, \sigma^2 I) dx_0.$$

Dividing by $p_\sigma(x)$,

$$\nabla_x \log p_\sigma(x) = \frac{1}{\sigma^2} \mathbb{E}[x_0 - x \mid x] = \frac{\mathbb{E}[x_0 \mid x] - x}{\sigma^2}.$$

Identifying the MMSE denoiser $D^*(x; \sigma) = \mathbb{E}[x_0 \mid x]$ gives Tweedie’s formula

$$\boxed{\nabla_x \log p_\sigma(x) = \frac{D^*(x; \sigma) - x}{\sigma^2}.} \quad (\text{C.6})$$

Because the Bayes-optimal denoiser is the conditional expectation, score matching and denoising regression are equivalent as learning objectives: fitting D_θ to minimise $\mathbb{E} \|D_\theta(x; \sigma) - x_0\|^2$ simultaneously learns an estimate of the score via (C.6).

C.6 Probability-Flow ODE from the VE-SDE

The variance-exploding forward SDE is

$$dx = g(\sigma) dW, \quad g(\sigma) = \sqrt{\frac{d\sigma^2}{dt}},$$

where we indexed time by σ . The corresponding Fokker–Planck equation for the marginal $p_t(x) = p_\sigma(x)$ is

$$\frac{\partial p_\sigma}{\partial t} = \frac{1}{2} g(\sigma)^2 \Delta_x p_\sigma.$$



For the deterministic PF-ODE $dx/dt = f(x, \sigma)$ the continuity equation is

$$\frac{\partial p_\sigma}{\partial t} = -\nabla_x \cdot (f p_\sigma).$$

Requiring both to produce the same marginals and rearranging,

$$-\nabla_x \cdot (f p_\sigma) = \frac{1}{2} g^2 \Delta_x p_\sigma = \frac{1}{2} g^2 \nabla_x \cdot (\nabla_x p_\sigma) = \frac{1}{2} g^2 \nabla_x \cdot (p_\sigma \nabla_x \log p_\sigma),$$

which is solved by $f = -\frac{1}{2} g^2 \nabla_x \log p_\sigma$. Switching to σ as the time variable ($d/dt = (d\sigma/dt) d/d\sigma$ and $g^2 = 2\sigma d\sigma/dt$) yields the clean form

$$\boxed{\frac{dx}{d\sigma} = -\sigma \nabla_x \log p_\sigma(x).} \quad (\text{C.7})$$

Combining (C.7) with Tweedie (C.6) gives the denoiser-parameterised PF-ODE

$$\frac{dx}{d\sigma} = \frac{x - D^*(x; \sigma)}{\sigma}, \quad (\text{C.8})$$

which is the form that first-order (Euler / DDIM) and second-order (Heun) integrators actually evaluate at inference.

C.7 DDIM: Non-Markovian Forward and Deterministic Sampler



(author?) [21] parameterise a family of forward processes that share the marginals $q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$ but are not Markov:

$$q_\sigma(x_{t-1} | x_t, x_0) = \mathcal{N}(\mu_\sigma(x_t, x_0), \sigma_t^2 I),$$

with $\sigma_t \in [0, \tilde{\beta}_t]$ a free parameter and μ_σ to be determined.

Determining μ_σ . Require that the induced $q_\sigma(x_t | x_0)$ matches the DDPM marginal $\mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$. The ansatz

$$\mu_\sigma(x_t, x_0) = \sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{\sqrt{1 - \bar{\alpha}_t}}$$

satisfies this requirement: write $x_{t-1} = \mu_\sigma + \sigma_t \xi$, substitute $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$, and compute

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \epsilon + \sigma_t \xi.$$

The noise $\sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2}\epsilon + \sigma_t \xi$ has variance $1 - \bar{\alpha}_{t-1}$, matching the DDPM marginal at time $t - 1$. The training objective (C.5) depends only on the marginals, so a model trained under DDPM can be sampled under any member of this non-Markovian family.

Deterministic limit. Setting $\sigma_t = 0$ removes the per-step noise entirely and gives the DDIM update

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0(x_t, t) + \sqrt{1 - \bar{\alpha}_{t-1}} \frac{x_t - \sqrt{\bar{\alpha}_t} \hat{x}_0(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}, \quad (\text{C.9})$$

with $\hat{x}_0(x_t, t) = (x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)) / \sqrt{\bar{\alpha}_t}$.

Variance-exploding form. In the VE normalisation $x_\sigma = x_0 + \sigma \epsilon$, the DDPM coefficients become $\sqrt{\bar{\alpha}_t} \rightarrow 1$, $\sqrt{1 - \bar{\alpha}_t} \rightarrow \sigma_t$, and Equation (C.9) simplifies to

$$x_{\sigma'} = \hat{x}_0 + \sigma' \frac{x_\sigma - \hat{x}_0}{\sigma} = x_\sigma + (\sigma' - \sigma) \frac{x_\sigma - \hat{x}_0}{\sigma}. \quad (\text{C.10})$$

Recognising $(x_\sigma - \hat{x}_0) / \sigma = -\sigma \nabla_x \log p_\sigma$ as the PF-ODE velocity (C.8), Equation (C.10) is exactly one step of explicit Euler on the PF-ODE. DDIM is therefore a first-order ODE integrator of the same flow that EDM’s Heun sampler integrates to second order — and the curvature of that flow is exactly what the straight-line FlowCast path of Appendix C.12 removes.

C.8 Equivalence of ϵ , x_0 , and v Parameterisations

Under the noising relation $x_t = \alpha_t x_0 + \sigma_t \epsilon$ (with $\alpha_t^2 + \sigma_t^2 = 1$ in the variance-preserving case and $\alpha_t = 1$ in the VE case), a network predicting one of $\{x_0, \epsilon, v\}$ can be converted linearly to the other two. (author?) [18] define

$$v = \alpha_t \epsilon - \sigma_t x_0, \quad (\text{C.11})$$

which gives the inverse relations

$$x_0 = \alpha_t x_t - \sigma_t v, \quad \epsilon = \sigma_t x_t + \alpha_t v.$$



These linear maps commute with the MSE loss up to a σ -dependent weighting, so choosing one parameterisation rather than another does not change the optimum, only the conditioning of the optimiser. EDM uses x_0 -prediction (via the preconditioning below); the v -prediction of (author?) [18] improves numerical stability near $\sigma \rightarrow 0$ and $\sigma \rightarrow \sigma_{\max}$; DDPM's $\mathcal{L}_{\text{simple}}$ uses ϵ -prediction. The EDM diffusion baseline of this thesis (Chapter 3) operates on x_0 -prediction; FlowCast, by contrast, regresses a velocity field directly and does not use any of these denoiser parameterisations.

C.9 EDM Preconditioning Derivation

(author?) [8] parameterise the denoiser through an inner network F_θ with skip, input, output, and noise scalings:

$$D_\theta(x; \sigma) = c_{\text{skip}}(\sigma) x + c_{\text{out}}(\sigma) F_\theta(c_{\text{in}}(\sigma) x, c_{\text{noise}}(\sigma)).$$

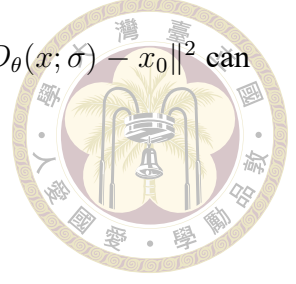
We derive $(c_{\text{skip}}, c_{\text{in}}, c_{\text{out}})$ from three requirements.

Requirement 1: unit-variance input to F_θ . The network input $c_{\text{in}}(\sigma) x = c_{\text{in}}(\sigma)(x_0 + \sigma \epsilon)$ has variance $c_{\text{in}}(\sigma)^2(\sigma_{\text{data}}^2 + \sigma^2)$. Setting this to 1 gives

$$c_{\text{in}}(\sigma) = \frac{1}{\sqrt{\sigma^2 + \sigma_{\text{data}}^2}}. \quad (\text{C.12})$$

Requirement 2: unit-variance target for F_θ . The squared loss $\|D_\theta(x; \sigma) - x_0\|^2$ can be written in terms of the network's effective target:

$$F^*(x; \sigma) = \frac{x_0 - c_{\text{skip}}(\sigma) x}{c_{\text{out}}(\sigma)}.$$



For the F_θ -regression loss to remain well-conditioned across σ , $\text{Var}(F^*) = 1$:

$$\begin{aligned} \text{Var}(F^*) &= \frac{1}{c_{\text{out}}^2} \text{Var}(x_0 - c_{\text{skip}} x) \\ &= \frac{1}{c_{\text{out}}^2} \left[(1 - c_{\text{skip}})^2 \sigma_{\text{data}}^2 + c_{\text{skip}}^2 \sigma^2 \right]. \end{aligned}$$

Setting this equal to 1 is one equation in two unknowns ($c_{\text{skip}}, c_{\text{out}}$); we need the third requirement to pin them down.

Requirement 3: minimise F^* 's sensitivity to x . (author?) [8] choose c_{skip} to minimise the expected squared magnitude of F^* :

$$\mathbb{E}\|F^*\|^2 \propto (1 - c_{\text{skip}})^2 \sigma_{\text{data}}^2 + c_{\text{skip}}^2 \sigma^2.$$

Differentiating in c_{skip} and setting to zero,

$$-2(1 - c_{\text{skip}}) \sigma_{\text{data}}^2 + 2c_{\text{skip}} \sigma^2 = 0 \implies c_{\text{skip}}(\sigma) = \frac{\sigma_{\text{data}}^2}{\sigma^2 + \sigma_{\text{data}}^2}.$$

Substituting back into the unit-variance requirement,

$$c_{\text{out}}^2 = \left(\frac{\sigma^2}{\sigma^2 + \sigma_{\text{data}}^2} \right)^2 \sigma_{\text{data}}^2 + \left(\frac{\sigma_{\text{data}}^2}{\sigma^2 + \sigma_{\text{data}}^2} \right)^2 \sigma^2 = \frac{\sigma^2 \sigma_{\text{data}}^2}{\sigma^2 + \sigma_{\text{data}}^2},$$

so

$$c_{\text{skip}}(\sigma) = \frac{\sigma_{\text{data}}^2}{\sigma^2 + \sigma_{\text{data}}^2}, \quad c_{\text{out}}(\sigma) = \frac{\sigma \sigma_{\text{data}}}{\sqrt{\sigma^2 + \sigma_{\text{data}}^2}}. \quad (\text{C.13})$$


Noise scaling. $c_{\text{noise}}(\sigma) = \frac{1}{4} \log \sigma$ is an empirically-motivated choice to equalise the dynamic range of the time-embedding input across $\sigma \in [\sigma_{\min}, \sigma_{\max}]$ (the log compresses orders of magnitude; the $\frac{1}{4}$ puts typical values in a band that matches the embedding frequencies).

Loss weighting. The EDM loss is

$$\mathcal{L}_{\text{EDM}} = \mathbb{E}_{\sigma} \left[\lambda(\sigma) \|D_{\theta}(x_0 + \sigma \epsilon; \sigma) - x_0\|^2 \right].$$

Substituting $D_{\theta} = c_{\text{skip}}x + c_{\text{out}}F_{\theta}(\cdot)$ and the target F^* , the inner squared error is $c_{\text{out}}^2 \|F_{\theta} - F^*\|^2$. To make the F_{θ} -regression effectively unit-weighted, we choose

$$\lambda(\sigma) = \frac{1}{c_{\text{out}}(\sigma)^2} = \frac{\sigma^2 + \sigma_{\text{data}}^2}{\sigma^2 \sigma_{\text{data}}^2}. \quad (\text{C.14})$$

With the Karras log-normal sampling $\log \sigma \sim \mathcal{N}(P_{\text{mean}}, P_{\text{std}}^2)$, the training-time distribution of σ combined with (C.14) concentrates the learning signal at mid-range σ , which is where the denoiser has the highest leverage on the final sample quality. This concludes the derivation; the five preconditioning scalars $(c_{\text{skip}}, c_{\text{in}}, c_{\text{out}}, c_{\text{noise}}, \lambda)$ of Equation (2.3) of the main text are now all justified from first principles.



C.10 Karras ρ -Spaced Noise Schedule

The sampling schedule $\{\sigma_i\}$ is chosen to equalise truncation error per step of a first-order ODE integrator of Equation (C.8). Under the ansatz

$$\sigma_i = \left(\sigma_{\max}^{1/\rho} + \frac{i}{N-1}(\sigma_{\min}^{1/\rho} - \sigma_{\max}^{1/\rho}) \right)^\rho,$$

the change of variables $u = \sigma^{1/\rho}$ makes the index i linear in u , and the per-step truncation error of Euler integration is approximately constant in u for ρ tuned to the curvature of the trajectory. **(author?)** [8] show empirically that $\rho = 7$ gives the lowest terminal FID among $\rho \in \{1, 3, 5, 7, 9\}$ across diverse datasets; we adopt it unchanged for the StormCast diffusion baseline, following **(author?)** [12].

C.11 CFM Theorem: Conditional = Marginal Gradient

Fix a Gaussian conditional probability path $p_t(x \mid z)$ indexed by a latent $z \sim q(z)$, with corresponding conditional vector field $u_t(x \mid z)$. The marginal flow-matching loss is

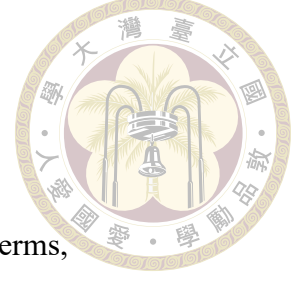
$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, p_t(x)} [\|v_\theta(x, t) - u_t(x)\|^2],$$

where $p_t(x) = \mathbb{E}_{q(z)}[p_t(x \mid z)]$ and $u_t(x) = \mathbb{E}[u_t(x \mid z) \mid x, t]$ is the marginal vector field.

This loss is intractable because $u_t(x)$ requires marginalising over z . The conditional flow-matching loss replaces the marginal target with the conditional one:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, z, p_t(x|z)} [\|v_\theta(x, t) - u_t(x \mid z)\|^2].$$

Theorem. $\nabla_{\theta} \mathcal{L}_{\text{FM}}(\theta) = \nabla_{\theta} \mathcal{L}_{\text{CFM}}(\theta)$.



Proof. Expanding the squared norms and dropping θ -independent terms,

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, p_t(x)} [\|v_{\theta}(x, t)\|^2 - 2 v_{\theta}(x, t)^{\top} u_t(x)] + \text{const},$$

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, z, p_t(x|z)} [\|v_{\theta}(x, t)\|^2 - 2 v_{\theta}(x, t)^{\top} u_t(x | z)] + \text{const}.$$

The first term is identical after applying the law of total expectation, $\mathbb{E}_{z, p_t(x|z)} = \mathbb{E}_{p_t(x)}$.

For the second term,

$$\begin{aligned} \mathbb{E}_{t, z, p_t(x|z)} [v_{\theta}(x, t)^{\top} u_t(x | z)] &= \mathbb{E}_t \mathbb{E}_{p_t(x)} [v_{\theta}(x, t)^{\top} \mathbb{E}[u_t(x | z) | x, t]] \\ &= \mathbb{E}_{t, p_t(x)} [v_{\theta}(x, t)^{\top} u_t(x)], \end{aligned}$$

using $u_t(x) = \mathbb{E}[u_t(x | z) | x, t]$. Hence $\mathcal{L}_{\text{FM}} = \mathcal{L}_{\text{CFM}} + \text{const}$, and their gradients coincide. $\square$

The upshot is that training against the conditional target is an unbiased (and computable) estimator of gradients of the marginal objective, and the latter is exactly the flow-matching objective whose ODE transports $p_0 \rightarrow p_1$.

C.12 Independent CFM: Straight-Line Path and Target

Take $z = (x_0, x_1)$ with $x_0 \sim \mathcal{N}(0, I)$ and $x_1 \sim p_{\text{data}}$, independent. Define the Gaussian probability path

$$p_t(x | x_0, x_1) = \mathcal{N}((1-t)x_0 + tx_1, \sigma_{\text{path}}^2 I), \quad t \in [0, 1].$$

Target vector field. Under the flow $\phi_t(x \mid x_0, x_1) = (1 - t)x_0 + tx_1 + \sigma_{\text{path}} \eta$ with η fixed, the associated vector field is

$$u_t(x \mid x_0, x_1) = \frac{d}{dt} \phi_t = x_1 - x_0,$$

which is independent of x (and of t) — a straight-line constant-velocity interpolation from noise to data. Plugging into $\mathcal{L}_{\text{CFM}}$,

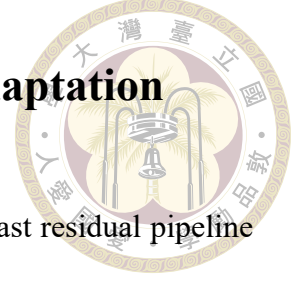
$$\mathcal{L}_{\text{I-CFM}}(\theta) = \mathbb{E}_{t, x_0, x_1, \eta} [\|v_\theta((1 - t)x_0 + tx_1 + \sigma_{\text{path}}\eta, t) - (x_1 - x_0)\|^2]. \quad (\text{C.15})$$

Why $\sigma_{\text{path}} > 0$. At $\sigma_{\text{path}} = 0$ the conditional path is a Dirac at each time, supported on the line segment between x_0 and x_1 . In the marginalised view this is a measure-zero set, so the score of the marginal $p_t(x)$ is undefined on that set and the empirical loss gradient is extremely high-variance in high dimensions (Rectified-Flow pathology; see 11). Taking $\sigma_{\text{path}} > 0$ convolves the path with a spherical Gaussian of width σ_{path} , which regularises the target field onto a thin tube around the interpolant and smooths the loss landscape. (author?) [24] report $\sigma_{\text{path}} \in [10^{-3}, 10^{-1}]$ all work in practice; FlowCast and our adaptation use $\sigma_{\text{path}} = 0.01$.

Compared to diffusion. The straight-line ODE of I-CFM and the PF-ODE of a score-based diffusion both transport $\mathcal{N}(0, I) \rightarrow p_{\text{data}}$, but their trajectories differ. The diffusion PF-ODE is curved: its Jacobian sees the local score, which in turn depends on the local data density, and its integration error grows quickly as the step count shrinks. The I-CFM ODE has constant velocity $x_1 - x_0$ along each conditional trajectory, so the marginal field is the closest approximation (in L^2) to a linear transport available — which is exactly why a single Euler step is already competitive.



C.13 FlowCast’s Standardised-Residual Adaptation



The adaptation used in Chapter 3 applies I-CFM to the StormCast residual pipeline with the following changes to the textbook I-CFM of (C.15):

1. **Residual target.** x_1 is the standardised residual R_t/σ_{data} rather than the raw field. Standardisation absorbs the per-channel scale so the noise $x_0 \sim \mathcal{N}(0, I)$ is comparable in magnitude to the data target.
2. **Condition bundle.** v_θ is conditioned on $c_t = \text{concat}(X_{t-1}, M_t, I)$ (the low-resolution background S_t is omitted from the direct path and reaches the velocity field only through the regression mean M_t ; see Section 3.5). This keeps the FlowCast residual anchored to exactly the same M_t as the EDM diffusion baseline. No extra VAE encoder is inserted, unlike the original FlowCast which works in latent space.
3. **Time rescaling.** The flow time $t \in [0, 1]$ is multiplied by $\tau = 1000$ before entering the SongUNet time embedding, because the positional-embedding frequencies of the pretrained architecture were tuned for the EDM $\log \sigma$ range.
4. **Channel-weighted + spectral loss.** The pointwise MSE is the channel-weighted L2 norm $\|\cdot\|_{2,\beta}$ of Section 3.6, and an additional radial log-PSD regulariser is applied to the implied clean residual $\hat{x}_1 = x_t + (1 - t) v_\theta$ on the qpepre channel. Applying the regulariser on $\hat{x}_1$ rather than on v_θ keeps the spectrum interpretable as a physical field’s spectrum.
5. **Endpoint clamp.** Training samples $t \sim \mathcal{U}(t_\varepsilon, 1 - t_\varepsilon)$ with $t_\varepsilon = 10^{-5}$. At $t = 0$ or $t = 1$ the conditional path collapses to a point mass and the Monte-Carlo loss

gradient is unbounded; the clamp prevents numerical blow-ups at negligible bias cost.



The resulting training loop differs from I-CFM only in these five bookkeeping details; the underlying theorem (§C.11) and path derivation (§C.12) carry over unchanged, and the sampler (3.7) is the explicit Euler integrator of the straight-line ODE.